

Temas que se evalúan en el primer parcial

Unidad 1 completa (incluye paper No silver bullet)

- Introducción a la Ingeniería del Software. ¿Qué es?
- Estado Actual y Antecedentes. La Crisis del Software.
- Disciplinas que conforman la Ingeniería de Software.
- Ejemplos de grandes proyectos de software fallidos y exitosos.
- Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.
- Procesos de Desarrollo Empíricos vs. Definidos.
- Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software
- Ventajas y desventajas de c/u de los ciclos de vida.
- Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.
- Componentes de un Proyecto de Sistemas de Información.
- Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Unidad 2

- Gestión de Producto
- Requerimientos Ágiles
- User Stories
- Estimaciones Ágiles
- Framework Scrum

Unidad 3

- SCM

ISW – Teórico Parcial 1

Unidad 1 - Ingeniería de Software en Contexto

Preludio - Software:

Software: programas + documentación + herramientas de construcción. **No es solo código.**

El **software** es un set de programas en conjunto con toda la documentación que los define para colaborar con su uso, desarrollo y mantenimiento. Se incluyen también las distintas herramientas utilizadas para su construcción.

Textual de la diapositiva: Software, en general, es un set de programas y la documentación que acompaña.

El software es conocimiento representado en distintos niveles de abstracción:



Beautiful grafico hecho con ia...

Los 3 tipos de software:

- **System Software**: controlan y **gestionan el hardware** de las computadoras. Proporcionan también una interfaz para la interacción.
- **Utilitarios**: subconjunto de software de sistema que **proporciona utilidades o herramientas adicionales** para mejorar la funcionalidad y rendimiento del sistema operativo.
- **Software de Aplicación**: se utiliza **para realizar tareas específicas** en una computadora. Satisface a usuarios en necesidades puntuales.

Ingeniería de Software:

Disciplina que **se encarga de todos los aspectos de la producción de software**, desde la primera etapa de especificación del sistema hasta el mantenimiento posterior a la puesta en marcha.

Realidades sobre las que se sostiene la Ingeniería de Software:

- **Resulta más barato a largo plazo** utilizar técnicas y métodos de la ingeniería de software que desarrollar rápida y descuidadamente. **Es muy costoso cambiar y corregir el software una vez que ya está operando.**
- Se suele requerir que la producción sea rápida y económica, pero que los resultados sean confiables.
- **Es muy importante entender el problema** y las necesidades del cliente **antes de proponer una solución**
- El diseño es crucial para la calidad y el mantenimiento.
- Surge de la crisis del software de 1968:
 - Dificultades y Errores en:
 - La planificación
 - La estimación de costos
 - La productividad
 - La calidad del software
 - Baja eficacia de las empresas que producen software.
 - 1968: el término lo plantea Friedrich Bauer para recalcar la dificultad para generar software libre de defectos.
 - También lo uso Dijkstra antes.

Causas de la Crisis del Software:

- La evolución del hardware permitió la creación de sistemas más grandes, complejos y potentes. Pero este **salto en el desarrollo del hardware no fue acompañado por un salto en el desarrollo del software.**
 - Era difícil de evolucionar y definir sobre una **actividad cuyo producto es abstracto e intangible.**
- **Aumento la demanda** de sistemas complejos:
 - Los métodos existentes (tradicionales) de ingeniería no permiten adaptarse a la demanda y los cambios característicos de la industria con respecto al software, por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
 - Expectativas bajas: las empresas se conforman con el software “solo funcione”. No tienen en cuenta el valor agregado que el software debe aportar
- **Subestimación de la complejidad** del desarrollo
- La **adaptación a las necesidades del cliente**, usualmente cambiantes
- La **ausencia de disciplinas formales** que cubran todas las fases de la producción de software.

Problemas actuales del desarrollo de Software:

- La versión final no satisface a los clientes.
- No es fácil extender y adaptar los programas.
- Mala documentación.
- Mala calidad.
- Los tiempos y costos exceden lo presupuestado.
- **Demanda Creciente**

- **Expectativas bajas**

Disciplinas que conforman la Ing. de Software:

- **Disciplinas técnicas:**
 - Captura de requerimientos
 - Análisis de los mismos
 - Diseño del software
 - Implementación del software
 - Prueba de software
 - Despliegue del software
 - Documentación
 - Capacitación de los usuarios
- **Disciplinas de Gestión:**
 - Actividades de planificación, monitoreo y control del proyecto de desarrollo de software. Se utilizan, por ejemplo, metodologías Ágiles para esto.
- **Disciplinas de Soporte:**
 - Disciplinas transversales al software, que permiten verificar la integridad y calidad de un producto de software. Incluyen:
 - Gestión de Configuración de Software (**SCM**).
 - Toma de Métricas.
 - QA – Aseguramiento de Calidad.

Resumen Disciplinas:

- **Disciplinas Técnicas:** Comprenden las actividades directamente vinculadas a la construcción del software **como producto**.
- **Disciplinas de Gestión:** actividades de **planificación, monitoreo y control del proyecto** de desarrollo mediante metodologías formales de gestión.
- **Disciplinas de Soporte:** Acompañan transversalmente todo el ciclo de vida para verificar, **auditar y garantizar la calidad, integridad y trazabilidad del producto y sus cambios**.
 - **Transversalmente:** porque no son una etapa fija del desarrollo (como codificar o diseñar), sino que se aplican de forma continua e ininterrumpida

Software vs Manufactura:

- El software es menos predecible.
- El software no se produce en masa, casi ningún software es igual a otro.
- No toda falla es error.
- El software no se gasta.
- El software no es gobernado por las leyes de la física.

Comentado [BCDL1]: No creo q tomen esto

No Hay Balas de Plata de Brooks → Accidental vs Esencial.

- **Complejidad Accidental:** dificultades que acompañan la producción pero no son inherentes a ella.

- **Complejidad Esencial:** la construcción de estructuras conceptuales entrelazadas y altamente complejas. Algoritmos, datos, relaciones.

La idea: escribir código tiene sus dificultades, pero no sería el verdadero problema. La especificación, el diseño y la validación de la estructura conceptual son los lugares donde florece la “complejidad accidental” que enfrente el desarrollo.

El ensayo contrata el progreso de algo tangible como el hardware en comparación con el software, inherentemente complejo e intangible.

La bala de plata: solución mágica inexistente para los problemas esenciales del desarrollo de software.



Las 4 propiedades inherentes irreducibles **esenciales** del software complejo:

Son las 4 dificultades esenciales para la construcción de software:

- 1) **Complejidad:** el tamaño del software escala de forma no lineal, y con esto crece su complejidad.
 - a. Mientras en un edificio se repiten elementos como ventanas, puertas y paredes, en el software hay mucha más unicidad, y por ende, crecimiento en tamaño.
- 2) **Conformidad:** adaptarse a interfaces humanas arbitrarias y a sistemas sin principios unificadores. Adecuarse a los requerimientos
 - a. el software debe adaptarse a las reglas aleatorias de su entorno, como el hardware, sistemas antiguos, reglas de negocio cambiante, sistemas operativos...
- 3) **Mutabilidad:** es infinitamente maleable y constantemente sometido a la presión del cambio continuo y a largo plazo.
- 4) **Invisibilidad:** carece de representación espacial o geométrica. No puede dibujarse la totalidad del software en un plano.
 - a. se necesitan múltiples planos que pueden ser muy distintos como para superponerlos.

Algunas soluciones a la Complejidad Accidental:

Reducen la carga de trabajo administrativo y sintáctico del programador. Evitan lo mecánico. Tiene que ver con aquellos aspectos que dificultan la construcción del software, pero han sido solucionados a lo largo del tiempo

- **Lenguajes de programación de alto nivel:** permiten abstraer el proceso de construcción del software con independencia del hardware en el que se ejecutan. Elimina la complejidad propia de la máquina.
- **Tiempo compartido:** reducción de tiempos de respuesta de un software. Permite no desviar el foco de lo que se desea construir.
- **Entornos de programación unificados:** unen librerías, archivos de distintos formatos, frameworks...
- **POO:** abstrae tecnicismos.
- **IA:** hace tareas por el humano, pero implica que el humano enseñe.
- **Sistemas expertos**
- **Programación automática**
- **Programación gráfica**
- **Verificación de programas**
- **Enviroments and tolos**
- **Estaciones de trabajo poderosas**

Algunas soluciones a la Complejidad Esencial:

- Comprar antes que construir: explorar las soluciones antes de arrancar a realizar.
- Usar prototipos y refinar requerimientos: saber que construir.
- Desarrollo Incremental: ampliar. Tener un software con pocas funcionalidades, pero funcionando, es mejor que tener un avance de un software complejo que no funciona.
- Capacitar a las personas: ampliar.

Diagnosticando a la bestia: Esencia vs. Accidente	
Las ganancias pasadas en productividad vinieron de eliminar barreras artificiales. Esa era la parte fácil.	
El Accidente (Problemas ya resueltos)	La Esencia (Problemas incurables)
☞ Representación en lenguajes engorrosos. ☞ Restricciones severas de memoria. ☞ Tiempos largos de compilación. ☞ Procesamiento por lotes (Batch) vs. Interactivo. (Fácil de automatizar y acelerar)	1. Complejidad: Ninguna parte del software es igual a otra. 2. Conformidad: Debe interactuar con sistemas humanos arbitrarios e ilógicos. 3. Mutabilidad: La presión constante por cambiar; el software es infinitamente maleable. 4. Invisibilidad: No tiene una representación geométrica o física inherente.



Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Se denomina Ciclo de vida a una serie de pasos a través de los cuales un producto o proyecto progresa en el tiempo.

Es una **representación simplificada y genérica de un proceso** que define elementos (como actividades, productos del trabajo, tareas) y el flujo del proceso/trabajo. Especifica un **orden y relación** entre dichos elementos.

Son modelos genéricos de los procesos de software independientes de los procedimientos de cada actividad del ciclo de vida: el modelo de proceso establece *qué* etapas se deben seguir y en *qué orden*, pero no impone las herramientas, técnicas o metodologías específicas que utilizas para ejecutar cada una de esas etapas.

Los distintos modelos de proceso pueden incluir las mismas actividades, **pero cada uno**, desde una perspectiva particular, **pondrá más énfasis en algunas respecto a otras** y establecerá criterios para determinar cuando y en que orden suceden las tareas.

Entonces, los modelos de ciclo de vida especifican:

- Las **fases del proceso** (requerimientos, especificaciones, diseño, etc)
- **Orden** en el cual se llevan a cabo

¿Ciclo de vida proyecto > o < Ciclo de vida del producto?

Comentado [BCDL2]: Ciclo de Vida (Life Cycle): Es el **fenómeno real y temporal** por el cual pasa un sistema desde su concepción inicial hasta su retiro definitivo. Describe la "vida" completa del software a medida que evoluciona en el tiempo.

Modelo de Proceso (Process Model): Es la **representación abstracta y teórica** (la plantilla o marco) de cómo se deben organizar las actividades dentro del ciclo de vida. Especifica las fases, las reglas de transición, los productos de trabajo (*deliverables*) y el flujo de trabajo.

Ciclo de vida del proyecto y ciclo de vida del producto:

En términos de tiempo, el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto: el ciclo de vida del proyecto dura lo que dura el desarrollo de software, mientras el ciclo de vida del producto dura toda la vida útil del software (hasta que se deje de utilizar). El ciclo de vida del producto **dependerá del mercado** y sus necesidades.

Es posible que un producto tenga varios ciclos de vida de proyectos dentro de su ciclo de vida, debido a las actualizaciones que se le van realizando.

Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.



Nota:

Si hablamos de Duración Temporal: Ciclo de vida proyecto < Ciclo de vida del producto

Si hablamos de Alcance de Gestión: Ciclo de vida proyecto > Ciclo de vida del producto

Clasificación de los Ciclos de Vida:

Modelo Secuencial:

Dispone las actividades en forma lineal. El proyecto progresa a través de una **secuencia ordenada de pasos/fases** y una **actividad no puede iniciar sin que la precedente haya sido finalizada**. Al final de cada fase, se hace una revisión para definir si el software está listo para avanzar al siguiente paso.

Son modelos **dirigidos por documentos**, siendo la documentación el trabajo principal de cada fase.

Se suele aplicar para construir aquellos sistemas donde **los requerimientos se comprenden bien y son exhaustivos** ya que éstos se congelan al finalizar la etapa de toma de requerimientos. El trabajo principal del producto es la documentación del software entre las distintas fases.

Ayuda a **minimizar los gastos generales de planificación** porque se puede hacer toda la **planificación por adelantado**: no proporciona resultados tangibles hasta el final del desarrollo. La **documentación va indicando el progreso**, no entregables.

Es sencillo de comprender y utilizar, por lo que se sigue utilizando (¿who?).

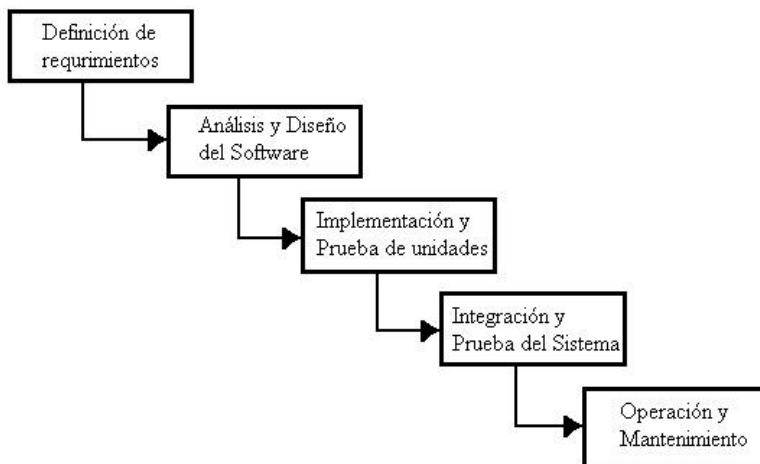
Se aplica en sistemas cuyos requerimientos son exhaustivos y se comprenden con claridad

Resumen: usar cuando hay una definición clara y estable del producto + metodologías técnicas claras.

Comentado [BCDL3]: exhaustivos significa que están especificados de forma **completa, detallada y sin omitir ningún aspecto** del sistema

Dificultades del modelo secuencial:

- Los proyectos raramente siguen un flujo secuencial.
- La incertidumbre es muy común en el desarrollo de software y este modelo no es muy bueno lidiando con ella, requiriendo que los requerimientos sean explícitos y estáticos, algo que rara vez suele suceder.
- El cliente no obtiene una versión funcional del producto hasta etapas muy avanzadas del proyecto.
- Encontrar un defecto implica grandes rediseños en etapas muy avanzadas.
- Dependencia entre los equipos de trabajo. Un equipo no puede comenzar hasta que el otro haya finalizado.
- Documentación burocrática y excesiva.



Modelo Iterativo/Incremental:

Aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software potencialmente entregable (serie de versiones). Los primeros incrementos incluyen las funciones más básicas y críticas, las que más requiere el cliente. Este enfoque vincula las actividades de especificación, desarrollo y validación.

Este es el ciclo de vida utilizado en metodologías ágiles.

Ventajas por sobre el modelo secuencial:

- Reduce el costo de adaptar los requerimientos cambiantes del cliente, menos retrabajo.
- De cada incremento se puede obtener una retroalimentación que permita aclarar las necesidades del cliente y para adaptar al producto conforme a estas.
- El cliente es parte del proceso de desarrollo -> P.O. por ejemplo.
 - En secuencial, el cliente debía esperar hasta la última instancia para recibir el producto, siendo incapaz de juzgarlo en su avance.

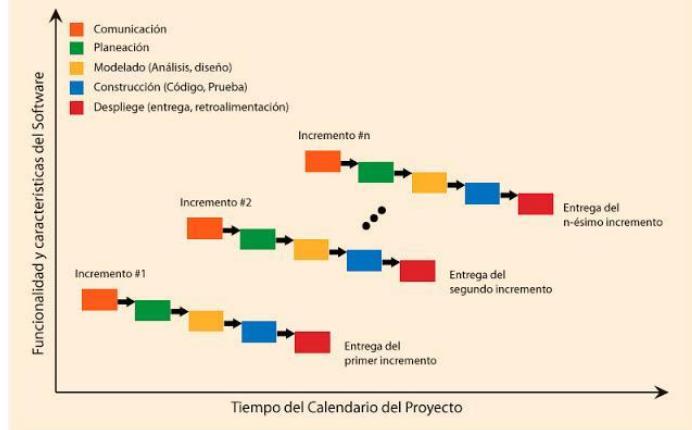
- Es posible que la entrega de software útil al cliente sea más rápida, considerando que en un principio no se incluirán todas las funcionalidades. El software podrá implantarse y generar valor mucho más temprano que en un proceso de cascada.
- Útil si trabajamos con **requerimientos cambiantes**.
- La **especificación, desarrollo y validación** están entrelazadas en lugar de separadas y aisladas



Dificultades desde una perspectiva administrativa:

- Invisibiliza el proceso: porque **prioriza el resultado** funcional visible (el "incremento") **sobre la documentación y la trazabilidad** lineal de cómo se construyó.
- Documentación cara en cada incremento.
- La estructura del software tiende a degradarse frente a una cantidad considerable de incrementos.

Modelo Incremental



Modelo Recursivo:

Es utilizado para **gestionar los riesgos del desarrollo a gran escala: sistemas complejos**. Se requiere de la intervención del cliente.

Cada recursión consta de 4 etapas:

- 1) Determinar **objetivos**.
- 2) Identificar y resolver **riesgos**.
- 3) **Desarrollar y probar**.
- 4) **Planificar la próxima** iteración.

La idea: **subdividir el proyecto en varios mini-proyectos**. Se intentan resolver en cada uno de estos los riesgos más relevantes hasta que no quede ninguno.

Se presentan prototipos que van mejorando continuamente: productos reutilizables independientes de la implementación. ¿?

Dificultades desde una perspectiva administrativa:

- La **reutilización** en distintos proyectos puede ser **costosa** en tiempo y dinero por el esfuerzo de adaptación.
- Se puede **carecer de mantenimiento** o documentación.
- Recién al final del ciclo tenemos una versión definitiva del producto.

Ejemplo: modelo espiral.

Administración de proyectos en relación con el ciclo de vida:

La elección de un ciclo de vida afecta de forma directa a la administración del proyecto, ya que cada modelo tiene un enfoque distinto.

Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos. Se elige un ciclo de vida conveniente según el trabajo a realizar.

La administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida. Pensemos en lo distinto que es gestionar un proyecto iterativo en comparación con uno tradicional.

Procesos de Desarrollo de Software:

- **Proceso:** secuencia de pasos ejecutados para un propósito dado.
- **Proceso de Software:** conjunto de actividades, métodos, prácticas y transformaciones que se utilizan para desarrollar o mantener el software y sus productos asociados.

Fuente: sw-CMM Actividades del proceso de desarrollo (las que conocemos de siempre):

- Especificación de requerimientos
- Análisis
- Diseño
- Implementación
- Prueba

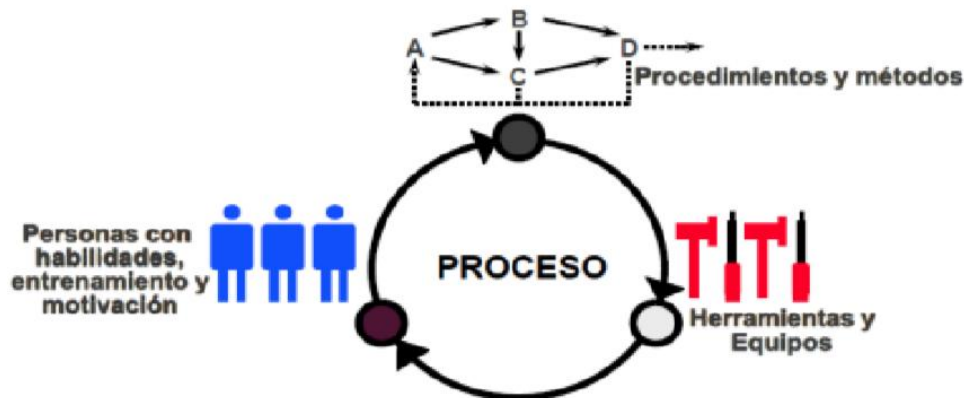
Los ciclos de vida definen el orden y la relación entre estas actividades.

Se tiene tres factores determinantes de los procesos:

- **Procedimientos y Métodos:** definir actividades, adaptar procesos al contexto y a la implementación, documentación de los mismos.
 - o Procedimientos escritos y definidos.
- **Personas:** motivadas, capacitadas y con habilidades para realizar las tareas asignadas. El software es una actividad humana-intensiva.
- **Herramientas y equipos:** materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Automatización de la mayor cantidad de tareas posible.

No existe un proceso ideal, el software es muy complejo como para que exista un solo proceso que abarque todo. Varias empresas han diseñado su propios procesos de software según los tres factores o según las características del software que se está desarrollando.

Comentado [BCDL4]: Mejorar



Distintos sistemas implican distintos procesos de desarrollo (lo que decíamos de elegir el ciclo de vida), por ejemplo:

- Para algunos sistemas, como los sistemas críticos, se requiere de un proceso de desarrollo muy estructurado (proceso definido).
- Para los sistemas empresariales, con requerimientos rápidamente cambiantes, es probable que sea más efectivo un proceso menos formal y flexible (filosofía ágil).

Consideraciones:

- 1) **Las actividades varían dependiendo de la organización** y el tipo de sistema a desarrollar.
 - i. Debe incluirse: productos, roles, responsabilidades y condiciones.
- 2) **Para administrar un proceso, antes hay que administrarlo**
- 3) **Un mismo proceso de desarrollo puede aplicarse a varios ciclos de vida.** La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.
- 4) Las personas utilizan herramientas y equipos para llevar a cabo los procedimientos que componen el proceso de desarrollo.

Comentado [BCDL5]: ampliar

Actividades Fundamentales:

- 1) **Especificación:** los clientes **definen**, junto al equipo, **el software** y sus restricciones.
- 2) **Desarrollo:** **diseño y programación** del software.
- 3) **Validación:** **es lo que el cliente quiere??**
- 4) **Evolución:** **se modifica el software para reflejar los requerimientos** cambiantes del cliente y mercado.

Procesos de Desarrollo Empíricos vs. Definidos

Procesos Definidos

Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de que cambie el equipo.

Son **deterministas**, ante la misma entrada pretenden obtener la misma salida: como una línea de producción. La predictibilidad del proceso permite que este pueda repetirse, permitiendo prevenir el esfuerzo, el riesgo o los costos. Repetirse → prevenir.

Los procesos definidos pueden usar cualquiera de los tres ciclos de vida vistos.

En la realidad, no es posible obtener el mismo resultado ante las mismas entradas, ni siquiera con el mismo equipo, ya que la salida **dependerá sumamente del contexto**, momento y personas.

La administración y control del proceso, en muchos de los casos, toman más importancia que el avance del software en sí (más importante la documentación del avance, que el avance en sí)

Comentado [BCDL6]: Ampliar

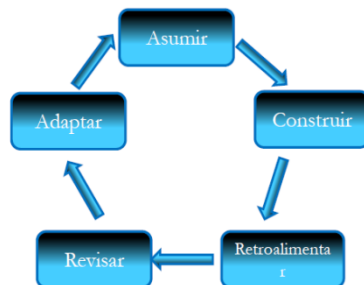


Procesos Empíricos

Se conforman en base a la experiencia interna y externa de las personas que intervienen en el proceso bajo determinado contexto. Son **no deterministas**: no asume que al aplicar un mismo proceso se obtendrán las mismas salidas/resultados. Se pone foco en el **factor diferencial que impone el contexto**.

Los Procesos de este tipo:

- 1) Se basan en **ciclos cortos de inspección y adaptación: retroalimentación**. Buscan adaptarse a dominios complejos mediante el análisis/revisión.



- 2) La administración y el control se dan por medio de **inspecciones frecuentes y adaptaciones para lograr buenas prácticas.**
- 3) Solo se recomienda **combinarlo con el ciclo de vida iterativo-incremental.**

La experiencia de un proyecto no es extrapolable a otro, los contextos son claves.

Los procesos empíricos se sostienen en la transparencia de la información entre integrantes de un mismo equipo.

Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software

Comentado [BCDL7]: Capítulo 7 de Desarrollos de proyectos informáticos (Rapid Development) de McConell

Criterios para la elección de ciclos de vida:

En función de las necesidades del proyecto y las características del producto.

- Es necesario evaluar un par de criterios antes de elegir un ciclo de vida para seguir:
 - Nivel de riesgo de los requerimientos
 - Claridad y cantidad de requerimientos
 - Los conocimientos técnicos del equipo de desarrollo
 - Los plazos que se tienen para ejecutar el proyecto
 - Posibilidad de validar resultados con el cliente

En que situación conviene cada modelo:

- **Secuencial:** **requerimientos claros y exhaustivos**, en un contexto de **baja incertidumbre**. El cliente **no necesita obtener un producto rápidamente**.
- **Iterativo e Incremental:** si, por el contrario, el cliente quiere obtener **resultados lo antes posible** y el entorno es cambiante.
- **Recursoivo:** disminuir riesgos en el **desarrollo de sistemas grandes y complejos**.

Ciclo de Vida	Procesos de desarrollo que lo admiten	Características que lo hacen elegible
Secuencial	Definidos	• Alta Certeza = Baja Incertidumbre • Forma de trabajo tradicional • No hay entregas parciales • Eliminación de riesgos de planificación
Iterativo e Incremental	Definidos o Empíricos	• Incetidumbre • Volatilidad de los requerimientos • Entregas parciales • En consecuencia, uso del producto desde etapas tempranas
Recursoivo	Definidos	• Control de riesgos elevado en cada iteración • Sin entregas parciales

Comentado [BCDL8]: congela el alcance desde el inicio y opera bajo un modelo predictivo

Diferentes modelos de ciclo de vida:

- **Code and Fix:**
 - Sin especificaciones, planificación o diseño.
 - Se corrige hasta que el cliente está satisfecho. Caro mantenimiento.
 - Ventaja:
 - No dedica tiempo a ninguna tarea que no sea codear.
 - Progreso inmediato
 - Sin planificación, documentación, control de calidad...
 - Requiere poca experiencia.
 - Útil proyectos pequeños y desechables
- **Cascada puro:**
 - Se sigue **una secuencia ordenada** de fases y se revisa al final de cada fase.
 - Las **fases no se superponen**.
 - Permite encontrar errores en etapas tempranas.
 - Documentación excesiva, resultados al final.
 - Se usa cuando:
 - la **definición del producto es completa y estable**
 - Los requerimientos de calidad dominan los costos y los tiempos
 - Funciona bien en equipos poco experimentados
 - **Inflexible.**
 - Puede pasar mucho tiempo sin entregar software funcional.
- **Cascada con fases solapadas**
 - Modelo de menor documentación.
 - **Superposición parcial de las fases** para acelerar el proceso.
- **Cascada de entrega por etapas**
 - Se ven **signos tangibles de progreso**.
 - Útil si **el cliente necesita funcionalidad inmediatamente**
 - También si las necesidades se modifican
 - Planificación rigurosa
- **Cascada con retroalimentación**
 - Variante donde **se puede volver a etapas anteriores** antes de llegar al final
- **Cascada con subproyectos**
 - Secuencial que **divide en subproyectos paralelos** (¿ok).
- **Modelo en Espiral**
 - Busca minimizar riesgos
 - Hace **análisis de riesgos sucesivos y busca alternativas constantemente**.
 - Al final de cada fase analiza la siguiente
 - Parte el proyecto en mini-proyectos
 - **Útil en proyectos grandes** para identificar riesgos
 - Proyectos largos y de diferentes equipos en cada ciclo
 - Muestra progreso al final
 - El riesgo disminuye con los incrementos, pero el análisis de riesgos es costoso y complejo

- Difícil es definir objetivos e hitos que identifiquen que se está listo para avanzar a la siguiente capa del espiral
- **Modelo Evolutivo**
 - **Se fija el tiempo o el alcance** mientras el otro se va modificando.
 - **Útil si los requerimientos no son claros.** Realiza entregas tempranas al cliente.
 - Modelo **recursivo** que repite tareas, dependiendo de la modificabilidad del sistema diseñado.
- **Diseño para cronograma**
 - **No se asegura de alcanzar la versión final: asegura la entrega** del producto en la fecha definida.
 - Para llegar con algo útil, prioriza los aspectos más importantes
 - Asegura tiempo, deja variable el alcance
- **Diseño para herramientas**
 - **Incluye solamente la funcionalidad soportada por las herramientas de software** existentes/disponibles en la organización.
 - Se usa en proyectos muy sensibles al tiempo.
 - Puede combinarse con otros ciclos de vida, pero no se podrán implementar todos los requerimientos.
- **Prototipación evolutiva**
 - **Se desarrolla el concepto del sistema a medida que se avanza.** Es útil cuando los desarrolladores no están seguros de que algoritmos o arquitectura optima utilizar.
 - Se da cuando hay cambios en los requerimientos.
 - Reduce la documentación, pero impide fijar calendarios o fechas de entrega precisas.

Componentes de un Proyecto de Sistemas de Información

Software en contexto



UN **PROYECTO** ES LLEVADO A CABO POR **PERSONAS** QUE IMPLEMENTAN **HERRAMIENTAS** PARA AUTOMATIZAR LOS **PROCESOS** Y QUE TIENE OBTIENE COMO RESULTADO UN **PRODUCTO**.

Proyecto

Un proyecto de software es un esfuerzo **temporal** que requiere de un acuerdo conjunto de especialidades y recursos **para la creación de un producto**, servicio o resultado **único**.

Es un **plan estructurado con inicio y fin** orientados a construir una solución digital específica.

Es una unidad organizativa de organizativa para adaptar al marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente. Necesita de personas, recursos y procesos para poder existir.

El proyecto es una instanciación del proceso definida en un ciclo de vida. opa.

Define el proceso y tareas que se van a realizar, **el personal** que se encargará de las actividades y **los mecanismos** que se implementarán **para valorar riesgos, controlar el cambio y evaluar la calidad**.

Tanto los proyectos como sus resultado son siempre únicos e irrepetibles.

Características de los Proyectos de Software:

- **Temporales**, de duración limitada. Se **termina cuando se alcanzan los objetivos**.
- **Orientado a un objetivo**: debo saber definir hacia dónde voy, con **alcances claros**.
 - **Objetivos no ambiguos, medibles y alcanzables**.
- **Son Únicos**: aunque algunos proyectos puedan ser parecidos, terminan siendo únicos porque tienen:
 - Siempre resultados distintos.
 - También contextos distintos.
- Tienen tareas relacionadas entre sí: **las tareas dependen unas de otras, se preceden entre sí y tienen asignados recursos y esfuerzos puntuales**.
 - Es clave que el proyecto tenga definidas algunas tareas a realizar y gestione al respecto.

Administración del Proyecto de Software: Project Management

Es la **organización y coordinación de personas y recursos** **que permiten desarrollar un producto** o servicio (de calidad!!!) **de acuerdo con el presupuesto, tiempo y funcionalidad acordadas con el cliente**.

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto y así, poder obtener como resultado el producto esperado.

La administración es una disciplina que **contiene dos grandes actividades**:

- **Planificación.**
- **Monitoreo y control.**

Una buena gestión no puede garantizar el éxito del proyecto, pero una mala gestión, generalmente, si implica la falla del proyecto.

Lo importante de la planificación es el acto de planificar, no el resultado. El acto de ponerme en el ejercicio de pensar tanto: riesgos, objetivos, alcances, estimaciones...

Metas más importantes de la administración de proyectos:

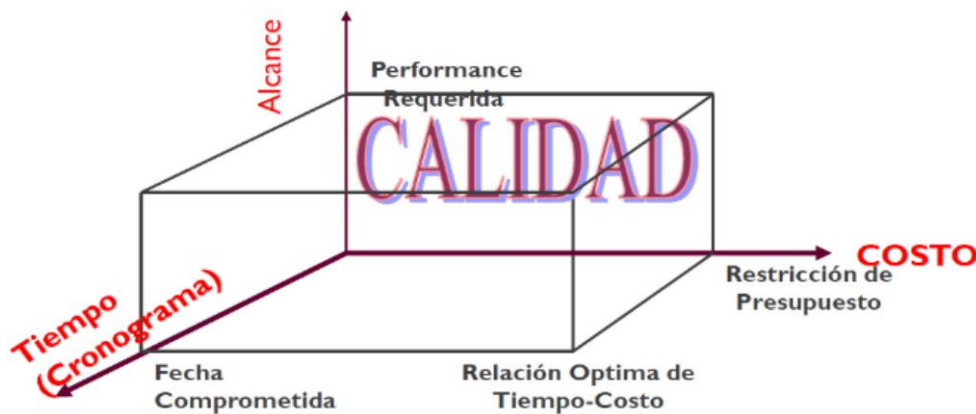
- 1) Entrega de software al cliente **en el tiempo acordado**.
- 2) Mantener los costos **dentro del presupuesto general**.

- 3) Entregar software que cumpla con las **expectativas del cliente**.
- 4) **Mantener un equipo** de desarrollo **óptimo y funcional**.

Atributos particulares / complejidades del Project Management:

- 1) **Producto intangible:** es **difícil medir el progreso** de un proyecto, en especial mientras no se obtenga un producto ejecutable. Es lo contrario a la construcción de un edificio, donde el progreso puede ser visible.
- 2) **Los proyectos de software suelen ser excepcionales:** **los conocimientos aprendidos en un proyecto son difícilmente extrapolables** a otro proyecto. Esto sucede, por ejemplo, por los distintos riesgos o las distintas tecnologías.
- 3) **Los procesos de desarrollo son variables y dependen de la organización:** a pesar del intento de **estandarizar los procesos de desarrollo**, la realidad es que **no es posible predecir** de manera confiable qué proceso es el adecuado para adoptar en el proyecto.

La Triple Restricción – Balance de los 3 factores



- **Alcance:** objetivos que el cliente expresa mediante requerimientos definidos.
 - En metodologías tradicionales: **se lo fija** primordialmente.
 - En metodologías ágiles: **se lo va refinando, definiendo y adaptando en base a las retroalimentaciones** del cliente. O sea: **se lo negocia**.
- **Tiempo:** lo que debería llevar completar el proyecto.
 - Se define la **fecha en la que se compromete a finalizar y entregar el producto**.
 - **Es fijo en las metodologías ágiles**.
- **Costo:** costo en recursos y dinero.
 - equipamiento, infraestructura, equipos, salarios, entre otros.

Comentado [BCDL9]: Hasta donde llego

La Triple Restricción es una de las bases de la gestión **tradicional** de proyectos que plantea que **el balance de los 3 factores** CAT (Costo, **Tiempo** y **Alcance**) **afecta directamente a la calidad** del proyecto.

Un **proyecto de alta calidad** es aquel que **entrega el resultado requerido, en la fecha estipulada** y **consumiendo el presupuesto planificado**. Es responsabilidad del Project Management asegurar el

balance de estas variables para lograr la mayor calidad posible: cumplir con una no debería desfavorecer a las otras. **La calidad no se negocia**, por lo que el equilibrio entre los factores será esencial.

Las tres variables están **íntimamente relacionadas**, por lo que modificar una implicará hacer variar las otras.

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande.

El problema en la aplicación de esta visión en enfoques tradicionales es que, al utilizar procesos definidos con ciclos de vida en cascada, los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este. Estos cambios, generan una baja en la calidad del producto final ya que, que se sacrifican funcionalidades o bien, se saltan etapas.

Líder del Proyecto: Project Manager

En la gestión tradicional, **el equipo del proyecto está formado por el Líder de Proyecto y el Equipo propiamente dicho**.

El PM es quien **realiza la totalidad de la toma de decisiones**, excluyendo la opinión del equipo (eso cambia en ágiles). El líder **asigna las tareas** que deben realizar los integrantes del equipo, y además, **maneja todas las relaciones con todos los stakeholders** del proyecto.

El Líder es una **figura muy útil si los técnicos carecen de las habilidades necesarias para tratar con personas**, por lo que este rol deberá tener las siguientes aptitudes:

- 1) Motivación: al personal
- 2) Organización.
- 3) Innovación.
- 4) Empatía.
- 5) Comunicación.
- 6) Liderazgo.

Las fuertes **habilidades blandas** del líder no implican que este carezca de **habilidades duras**, este debe poseer **conocimientos sobre técnicas, herramientas y el proyecto** mismo y el producto que se está construyendo.

Características propias del líder:

- 1) **Resolución de problemas:** **tanto técnicos como organizativos**. Identifica los problemas y estructura su solución.
- 2) **Identidad administrativa:** confianza para asumir el control.
- 3) **Logro:** incentiva al equipo y a su iniciativa.
- 4) **Influencia:** define la estructura del equipo en función de la retroalimentación.

Responsabilidades del Líder de proyecto:

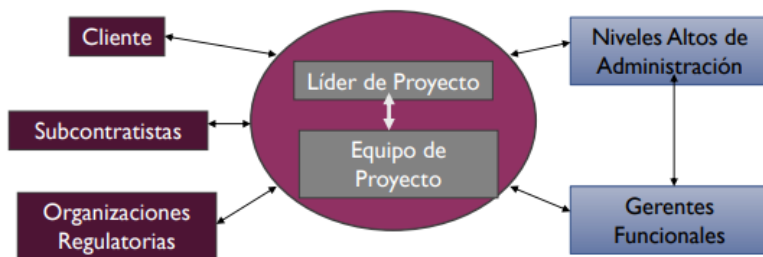
1. **Definir el alcance** del proyecto
2. **Identificar involucrados, recursos y presupuesto**

Comentado [BCDL10]: Todo esto después se explica con los triángulos

Comentado [BCDL11]: 1) Toma las decisiones
2) Asigna las tareas a los miembros
3) Maneja el vínculo con los stakeholders

3. Detallar tareas

4. Estimar tiempo y requerimientos
5. Identificar y evaluar riesgos
6. Preparar planes de contingencia
7. Controlar hitos y reversiones
8. Administrar el proceso de control de cambios
9. Producir reportes de estado



Equipo del Proyecto

Grupo de personas **comprometidas con alcanzar un conjunto de objetivos** compartidos, por los que se sienten **mutuamente responsables**.

Entre sus **características** se encuentran:

- 1) **Diversos conocimientos y habilidades.**
Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y características propias, al igual que sus personalidades e ideas.
- 2) Usualmente son un **grupo pequeño**, permitiendo una **comunicación efectiva**.
- 3) Posibilidad de trabajar juntos efectivamente / desarrollar sinergia
Trabajan en forma conjunta, buscando un equipo maduro y estable. Se evita que los integrantes estén cambiando constantemente.
- 4) Tienen **sentido de responsabilidad como unidad** y se focalizan en el cumplimiento de las metas grupales.



Stakeholders

Son los interesados en el proyecto.

Google: cualquier persona, grupo u organización que tiene un interés, impacto o vínculo en una empresa, proyecto o decisión corporativa. Pueden ser **internos o externos**.

Plan de Proyecto

Artefacto de gestión que documenta:

- ¿**Qué** es lo que hacemos? -> **alcance**
- ¿**Cuándo** lo hacemos? -> **calendario**
- ¿**Cómo** lo hacemos? -> **tareas, ciclo de vida y recursos**
- ¿**Quién** lo va a hacer? -> **división de tareas**

Es como una hoja de ruta.

Básicamente, documenta las decisiones que se toman a lo largo del desarrollo en términos de:

- El proceso
- La división de las tareas a realizar
- La asignación de responsabilidades
- Equipos de trabajo
- Calendario de desarrollo

Actividades de soporte: planes de contingencia, evaluaciones de calidad, riesgos... O sea:

1) Definición del Alcance del proyecto

Cosas Distintas:

Alcance del Producto: todas las características/funcionalidades que pueden incluirse en un producto o servicio.

Se mide contra la ERS.

Alcance del Proyecto: Es todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.

Se mide contra el plan del proyecto.

El alcance del proyecto, junto con el objetivo del mismo, responde a **qué se está desarrollando**. (definiciones del PMI)

- > El alcance del proyecto puede cambiar con el objetivo, pero esto no significa que cambie también el alcance del producto.
- > Por otro lado, que cambie el alcance del producto sí condiciona un cambio en el alcance del proyecto.

El alcance del proyecto es más amplio y abarca al del producto

2) Definición del proceso y ciclo de vida

Al definir el proceso a utilizar, se responde a **cómo se va a desarrollar el proyecto** a partir de la información del contexto del desarrollo: objetivos, alcances, personas, recursos, clientes, forma de trabajo de personas, etc...

También se deben tener en cuenta las características y la complejidad del software a desarrollar, como el alcance y los objetivos.

El ciclo de vida del proyecto definirá cuando se debe hacer cada tarea y cuanto de esa tarea se puede hacer. Define:

- Trabajo técnico para realizar en cada fase**
- Involucrados** en cada fase
- Como **controlar y aprobar** cada fase
- Como **generar los entregables**
- Como **revisar, verificar y validar** el producto

En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental).

3) Estimación

Tradicionalmente, en un proyecto de desarrollo de software se trata de estimar (en el orden de mención):

- Tamaño del Producto:** tradicionalmente se estima utilizando los requerimientos tomados de la etapa de requisitos. Un aumento del tamaño trae consigo un aumento de la complejidad del software. El tamaño de un producto se puede estimar en muchas

escalas, por ejemplo: líneas de código, cantidad de requerimientos, story points, puntos de función ajustados, etc.

- b) **Esfuerzo**: se mide en cantidad de horas persona lineales. Conociendo el tamaño y sabiendo el esfuerzo varía según este, se deberá estimar cuantas horas hombre lineal se necesitarán para construir el producto. El esfuerzo influye significativamente en el costo del proyecto.
- c) **Calendario**: respondemos a cuándo se van a realizar cada una de las tareas definidas. No es necesario ser tan específico como plantear el día a día, sino que tratar de escalar las tareas con periodos como meses o trimestres. Es necesario realizar esta estimación para poder darle una idea al cliente de cuánto tiempo se va a demorar la finalización del proyecto. Se debe realizar esta estimación en base al tamaño y esfuerzo.
- d) **Costos**: depende directamente del tamaño y el esfuerzo que se necesite para el desarrollo de software. Es muy útil esta estimación para darle una idea al cliente de cuál es el costo aproximado del proyecto.

Comentado [BCDL12]: Horas persona sería lo q tradicionalmente se llamaba horas hombre

4) Gestión de riesgos

Riesgo: es la probabilidad de ocurrencia de una pérdida o daño que afecte directamente al proyecto y pueda comprometer el cumplimiento de su objetivo.

Problema o evento esperando para suceder...

Es de suma importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y realizar una cuantificación de estos, teniendo en cuenta la exposición del proyecto ante los mismos para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.

Clasificación de los riesgos:

- **Riesgos del proyecto**: aquellos que alteran al calendario o los recursos del proyecto.
 - Ejemplo: renuncia de un miembro experimentado
- **Riesgos del producto**: aquellos que afectan la calidad o el rendimiento del software a desarrollar.
 - Ejemplo: un componente adquirido no se desempeña como se esperaba.
- **Riesgos organizacionales o de negocio**: aquellos que afectan a la organización que desarrolla o adquiere el software a desarrollar.
 - Ejemplo: un competidor que introduce un nuevo producto.

Comentado [BCDL13]: Hay una formula rara capaz la copie

Ante los riesgos identificados, se pueden tomar 3 actitudes:

- **Negación**: hacer como que no existen. (malo!)
- **Gestión Reactiva**: esperar que el riesgo ocurra y recién ahí ver que hacer.
- **Gestión Proactiva**: generar planes de mitigación y contingencia sobre el riesgo.

La Gestión Proactiva parece lo mejor, pero también es importante definir a cuáles riesgos la voy a aplicar.

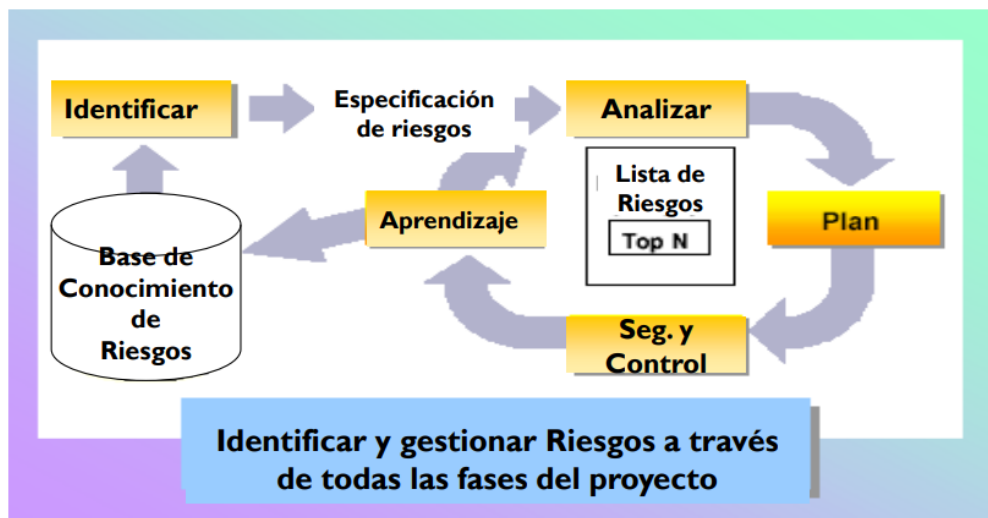
Comentado [BCDL14]: Probablemente no me va a dar la guita y el tiempo para aplicarla a todos los riesgos

Los planes de contingencia permiten actuar en caso de que un riesgo suceda, y poder volver a la normalidad lo antes posible.

Grandes riesgos en el desarrollo de software:

- cambios de requerimientos
- abandono de integrantes del equipo
- falta de capacidad
- desconocimiento de los avances tecnológicos.

Gestión del riesgo:



1. **Identificación de riesgos:** se identifican los riesgos que suponen una mayor amenaza para el proyecto, el producto a desarrollar o la organización.
2. **Análisis de riesgos:** para cada riesgo identificado se realiza un juicio acerca de la probabilidad y el impacto del mismo. El criterio es, por lo general, la experiencia.
3. **Planeación del riesgo:** por cada riesgo, se definen estrategias para manejarlos: acciones a tomar, desarrollar planes de contingencia, etc.
4. **Monitorización del riesgo y control:** proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado. Revalorar el riesgo.
5. Aprendizaje. 😊

5) Asignación de recursos y responsabilidades

La asignación de responsabilidades a personas permite **asignar roles a los integrantes del equipo** de trabajo. **Una persona puede tener más de un rol dentro del mismo proyecto.**

Se debe pensar en capacidades de los integrantes del equipo al asignar los roles o seleccionar el personal. Actividad humano-intensiva etc.

6) Programación de proyectos

En base a la estimación del calendario, se refina lo planteado para definir en detalla cada tarea que se debe cumplir en el desarrollo.

Se trata de detallar las tareas hasta descomponer los procesos en actividades a nivel días.

Se realiza con diagramas de Gantt:



7) Definición de métrica

Métrica: medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo.

Las métricas deben definirse de forma clara, sin ambigüedades.

Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software

En las metodologías tradicionales, acá se definen métricas que miden y exponen los avances. En cambio en metodologías ágiles, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración.

8) Definición de controles

En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

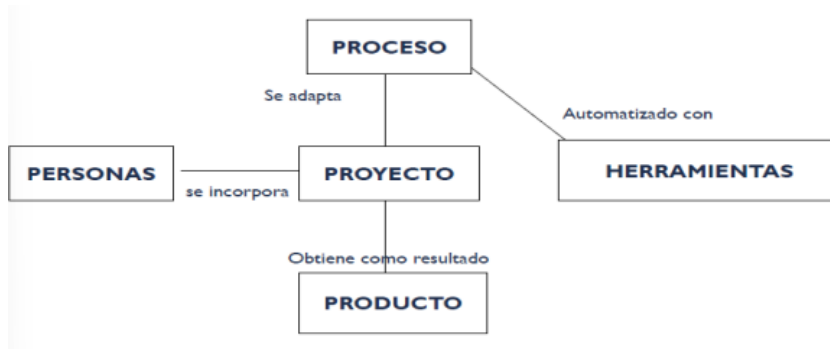
Causas de fracasos de proyectos

- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.

- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo

Vínculo entre Proceso, Proyecto y Producto

en la gestión de un proyecto de software.



El **proceso**, automatizado con **herramientas**, se adapta al **proyecto** que incorpora **personas** para obtener como resultado un **producto** o servicio.

Comentado [BCDL15]: El proceso se adapta al proyecto! No al revés

Sino también: UN **PROYECTO** ES LLEVADO A CABO POR **PERSONAS** QUE IMPLEMENTAN **HERRAMIENTAS** PARA AUTOMATIZAR LOS **PROCESOS** Y QUE TIENE OBTIENE COMO RESULTADO UN **PRODUCTO**.

Para obtener un resultado (ya sea producto o servicio), es necesario que el proyecto adopte un proceso de desarrollo que le permita definir las actividades, métodos, técnicas, prácticas y transformaciones que se utilizarán. El proceso elegido debe modificarse según sea necesario.

El proceso a utilizar dependerá del producto y su complejidad.

Los procesos deben ser adaptados a cada proyecto, no al revés. El proceso proporciona un marco conceptual en donde se establece un plan completo para las actividades

(y métodos y prácticas) **del desarrollo del software**. Como no existe una metodología universal para nuestra disciplina, **una correcta gestión** deberá lograr que el proceso se adapte al proyecto.

Las disciplinas de gestión se encargan de moldear y adecuar las actividades a las necesidades específicas del proyecto y del equipo de trabajo. Por su parte, las disciplinas de soporte son independientes del marco conceptual y operan de manera transversal, recubriendo todo el ciclo de vida del desarrollo para garantizar la calidad del software.

Un proyecto de desarrollo está integrado por un equipo de trabajo, dentro del cual deben estar los **roles bien definidos**, para que cada uno de los integrantes asuma la responsabilidad que le corresponda. El equipo va a basar su trabajo en el proceso adoptado, en el cual se van a tomar sólo aquellas actividades y formas de trabajo que deberán utilizar y seguir para lograr el desarrollo del producto/servicio deseado.

En el desarrollo de un proyecto, se busca utilizar diversas herramientas que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso adoptado. Por ejemplo, el uso de un software de diseño para realizar el análisis y diagramación de clases.

Unidad 2 – Gestión Lean-Agile de Productos de Software

Manifiesto Ágil

Es un documento redactado en 2001 por varios expertos en la Ingeniería de Software que **promueve la filosofía ágil para el desarrollo de software**, la cual abarca y se define en una serie de valores y principios.

Valores Ágiles



Priorizan ciertos aspectos del desarrollo por sobre otros.

1) **Individuos interactuando > Procesos y Herramientas**

- **Es preferible un equipo de personas motivadas**, capaces, portadoras de conocimiento y correctamente comunicadas entre sí **con herramientas pobres** antes que **un equipo disfuncional y desmotivado con las mejores herramientas**.
- El desarrollo de software es una **actividad humano-intensiva**.
 - depende fundamentalmente del intelecto, la creatividad, la comunicación y el juicio crítico de las personas.
- Los procesos ágiles intentan **capitalizar las fortalezas de cada individuo**
 - y mitigar sus debilidades
- No se busca homogeneizar al equipo
- **Los procesos** y herramientas deben ser un complemento para mejorar la eficiencia y éstos **se deben adaptar al equipo y no al revés**.

2) **Software Funcionando > Documentación Extensiva**

- Versiones estables, incrementales y mejoradas del software al finalizar cada iteración

- Tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos
 - para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente
- Solo se documenta aquello que agregue valor al producto centrándose en las necesidades del cliente
- **Sí se genera documentación, solo que no excesivamente**, ya que **los documentos no pueden ofrecer la generación de valor que se logra con la comunicación y retroalimentación.**

3) **Colaboración con el cliente > Negociación contractual**

- Mejor noción de los requerimientos de negocio y las necesidades del cliente
- **Integración con todas las partes interesadas** en el software para trabajar en conjunto por los mismos objetivos
- Cada miembro participando en la toma de decisiones > documentación extensiva
- Comprometer al cliente como parte importante del producto que se está construyendo.
- Pruebas y feedback, el cliente es cercano
- **Sí existen los contratos**, pero se intenta de que los términos y condiciones no generen una distancia entre cliente y equipo. **La rigidez del contrato no debe detener la adaptación ante cambios.**

4) **Responder a los cambios > Seguir un plan**

- **Es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan**, mucho menos al principio del desarrollo.
- Con el paso del tiempo **surgirán nuevas necesidades**
- Entorno cambiante para el software.
- Inspección y adaptación como valores heredados del empirismo.
- Visión positiva que **acepta los cambios** y asume su eventual aparición.
- Al finalizar cada iteración, la experiencia de desarrollar el incremento genera cierto conocimiento en el equipo, que afecta a la planificación en los distintos niveles.
- De forma similar, mostrar o desplegar esos incrementos a los usuarios finales, también generará nuevo conocimiento (principalmente por software) para esos procesos de planificación.
- O sea: **el nuevo conocimiento empujará a los cambios.**

Los 12 Principios del Manifiesto Ágil

1) **Satisfacer al cliente como mayor prioridad:** relacionado a los valores 3 y 4

El Product Owner (proveniente del lado del negocio) es quien **determina la prioridad e importancia de cada funcionalidad.**

En cada incremento del producto se busca retornar el mayor valor posible.

Se trabaja planteando las necesidades de los usuarios, por ejemplo, a través de las User Stories.

2) **Aceptar que los requisitos son cambiantes:** relacionado a los valores 3 y 4

El equipo ágil está abierto al cambio, no se deja atar por un plan sobre estructurado. El equipo debe ser **flexible y responder** ante las necesidades cambiantes de los cliente y/o usuarios.

3) **Entregar software funcional con frecuencia:** relacionado al valor 2

Es imprescindible que un equipo ágil, al finalizar cada iteración, haya sido capaz de transformar uno o más requerimientos en código desarrollado, testeado y potencialmente desplegable. Cumplimiento del DoD.

4) **Técnicos y no técnicos trabajan juntos durante todo el proyecto** relacionado al 1 y 3

Importancia del Product Owner que acompaña al desarrollo a pesar de ser externo al equipo. Este viene del lado del cliente, representado las necesidades e intereses de los usuarios a los que está destinado el fruto del desarrollo.

El PO, tras cada iteración, ayuda a marcar prioridades y guiar que requerimientos serán implementados y en que orden.

5) **Desarrollamos proyectos en torno a individuos motivados:** relacionado al valor 1

Los participantes deben estar motivados, comprometidos y percibir un objetivo común para todo el equipo. (ampliar esta hippeada

6) **El método más eficiente de comunicar es con conversaciones cara a cara:** valor 1

Base en la psicología: la comunicación no es solo palabras, sonido y texto, sino también gestos, expresiones y corporalidad.

7) **La mejor métrica de progreso es el software funcionando:** valor 2

El manifiesto ágil plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente

8) **Promover el desarrollo sostenible:** valores 2 y 4

Equipos estables y duraderos, formando relaciones continuas y confiables que mejoran la dinámica de trabajo y permiten alcanzar un mayor nivel de madurez y autoconocimiento (esto último es útil para las estimaciones).

9) **La atención continua a la excelencia técnica y al buen diseño mejora la agilidad:**

valor 2

Manifiesta que la calidad del producto no es un aspecto negociable. Las necesidades del cliente son lo primero que se tiene en mente.

10) **La simplicidad es esencial:** valor 2 y 3

Nos centramos en las necesidades que aportan un valor al cliente, sin perder tiempo en detalles que no requieren atención inmediata. No se hacen cosas que el cliente no haya pedido.

11) **Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados:**

valor 1

A diferencia de las organizaciones tradicionales y jerárquicas, donde los equipos solo acatan órdenes, en esta filosofía los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos, herramientas, sugerencias, etc... todo de utilidad...

Principio está sustentado por el empirismo, el cual establece que el conocimiento surge dentro del equipo de trabajo.

Los equipos tienen un mayor poder de toma de decisiones.

12) A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar:

valores 1 y 4

Se basa en **dos de los pilares de los procesos empíricos**:

- Inspección
- Adaptación

Un equipo ágil debe **inspeccionar** su propia forma de desempeño, **para adaptar aquellos aspectos que sean necesarios corregir**, mejorar o incorporar al equipo, de manera que el comportamiento del equipo como un todo mejore, y de esa forma se pueda responder a las necesidades cambiantes de los clientes.

Filosofía Ágil

El objetivo primordial es **entregar software de forma frecuente para adaptarse al entorno cambiante**, donde **los requerimientos varían excesivamente**.

Se busca involucrar al cliente en el desarrollo para obtener una **rápida y frecuente retroalimentación**, evitando que el desarrollo tome rumbos alejados de sus ideas y necesidades.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el **concepto del empirismo subyace al proceso** (es un pilar fundamental). En lugar de seguir un plan rígido teórico, **el proceso se construye y toma decisiones basándose en la experiencia real, la observación directa y los resultados medibles en cada iteración.**

El software no se desarrolla como una sola unidad, sino como una serie de incrementos, cada uno de ellos incorporando una nueva funcionalidad al sistema. Esta es una de las **diferencias con el PUD o el RUP**, que, **si bien utilizan un modelo de proceso iterativo - incremental, son procesos definidos.**

Generalmente **los incrementos se crean cada 2 o 3 semanas**, y **al finalizar, el resultado se pone a disposición del cliente.**

Los 3 Pilares del Empirismo:

1. **Transparencia**: se refiere a la **comunicación honesta y abierta entre involucrados**. Como el proyecto **es de todos, el conocimiento** también lo es. Se promueve la colaboración entre miembros y la **toma de decisiones descentralizada** pero bien informada y fundamentada. Importancia de un lenguaje común.
2. **Inspección**: se refiere a la **revisión regular y sistemática del progreso** de lo trabajado y los productos resultantes para detectar problemas y desviaciones del plan. La inspección permite una **evaluación temprana y continua del progreso** y ayuda a tomar decisiones informadas sobre los próximos pasos.
3. **Adaptación**: se refiere a la **capacidad de adaptarse rápidamente** (lo más inmediateamente posible) y **ajustar el trabajo frente a** las desviaciones y **problemas revelados por la inspección.**

Ágil no es:

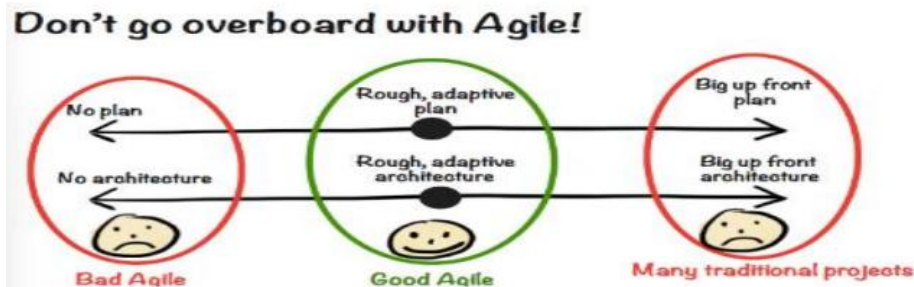
- Una metodología
- Un proceso
- Un framework completamente definido

Que es ágil y cuando es aplicable

Ágil es una ideología que guía el desarrollo del producto de software.

Los frameworks ágiles son distintos marcos de trabajo para implementar los principios y valores ágiles, generalmente orientados a la gestión de proyectos, no puntualmente el desarrollo.

Comentado [BCDL16]: revisar



Ser ágil != ser indisciplinado

Las metodologías ágiles son aplicadas para desarrollos donde la incertidumbre predomina y los contextos son complejos y cambiantes.

Ventajas del Ágil

- **Acelera la entrega** de productos que **aportan valor al cliente**
- Permite **adaptarse fácilmente a cambios de requerimientos**, o de las prioridades de estos
- Permite alinear los objetivos organizacionales y TI
- **Reduce riesgos**
- Pone foco en el equipo
- **Reduce costos**
- Mejora la visibilidad del proyecto



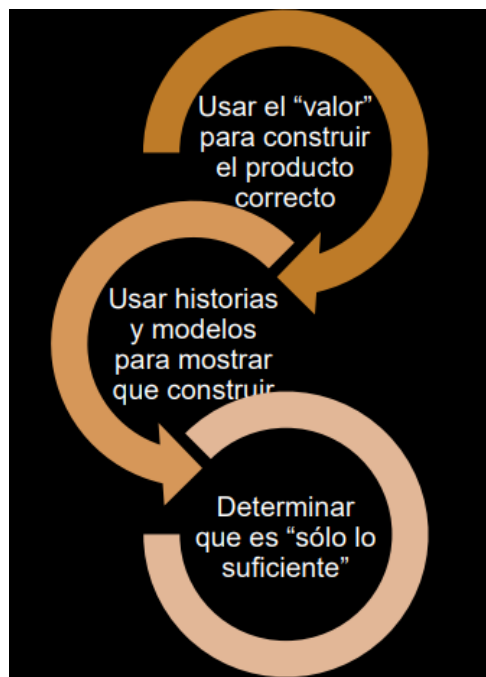
Requerimientos en Ambientes Ágiles y Desarrollo Ágil

Ágil, a diferencia de las metodologías tradicionales, **asume que los requerimientos se irán descubriendo con el avance del proyecto**. En un principio, se definirán algunos requerimientos suficientes para empezar a trabajar, pero la retroalimentación tardará en llegar. O sea: **existe la**

posibilidad de empezar a desarrollar el producto sin que el producto esté completamente definido.

Una forma de capturar y definir funcionalidades, de un nivel más alto que los clásicos requerimientos o que los casos de uso, **son las historias de usuario.**

Las US son descripciones breves enfocadas en la perspectiva y las necesidades del usuario utilizadas para definir el alcance de cada iteración del proyecto.



- **Usar el valor para construir el producto correcto:** **construimos a partir de las necesidades del cliente**, ya que el foco de ágil es aportar valor de negocio.
- **Determinar que es “sólo lo suficiente”:** los requerimientos deben ser encontrados de a poco, **no buscar todos los requerimientos desde un inicio y quedarnos solo con eso**. Se debe **asumir que se comienza desde un mínimo** de requisitos.
- **Usar historias y modelos para construir:** **se construye en conjunto con el cliente** que conozca el negocio y el producto. Por ejemplo: el PO escribe las US.

Entonces, resumiendo:

- La captura de requerimientos no solo se hace al principio del proyecto
- Se trata de hacerla involucrando al cliente.

La triple restricción – enfoque ágil – triángulo ágil

En las metodologías ágiles, el triángulo de hierro era una modelo para la gestión que planteaba tres variables estrechamente relacionadas: alcance, costo y tiempo. En este, cambiar una de las variables implicaba modificar a las otras dos.

En el enfoque ágil, se trata de **aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo**, y no es posible definirlos al comienzo de este. Se considera que, si bien las tres variables son flexibles, es mejor dejar variable solamente el tiempo. Entonces, **el enfoque trata de dejar el tiempo y los recursos fijos**, teniendo en cuenta “cuánto software” le puedo entregar en “cuanto tiempo” y con tal costo de desarrollo.

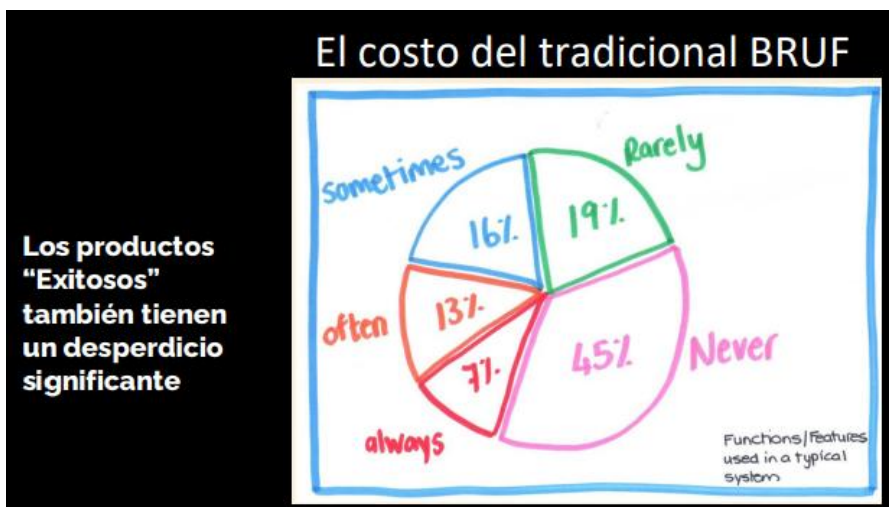
El triángulo ágil, como modelo, plantea que se fije el tiempo de entrega y los costos, manteniendo flexible, adaptable y renegociable el alcance. Es lo contrario al enfoque tradicional (triángulo de hierro), donde el alcance se fijaba, teniendo costos y tiempos de entrega variables.

Comentado [BCDL17]: Reusmen!

BRUFF – Big Requirements Up Front

Todos los productos de software, incluso los exitosos, tienen un desperdicio significativo: una buena parte de las funcionalidades desarrolladas nunca es utilizada por los usuarios.

La idea de “requerimientos grandes al frente”, es la de desarrollar primordialmente las funcionalidades de gran valor, uso e importancia para el cliente. Evitamos priorizar funcionalidades secundarias y potencialmente desechadas.



Las historias de usuario (US) se utilizan para definir el alcance de una iteración, y se priorizan en función del valor que aportan al producto. Es el PO quien va a identificar las necesidades y prioridades del negocio y, en consecuencia, priorizará los requerimientos del producto. Decidirá que se necesita primero, que tiene un mayor valor para el negocio. Para eso existe...

Product Backlog

Lista priorizada y ordenada donde los requerimientos se ordenan por importancia: los más cruciales arriba, los menos relevantes abajo. Se representa como una pila.

Comentado [BCDL18]: Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

LA GESTIÓN ÁGIL DE LOS REQUERIMIENTOS APUNTA A TRABAJAR Y A PONER EL ESFUERZO EN LO QUE NOS DA VALOR, Y A TRABAJARLO EN EL MOMENTO ADECUADO. O sea: lo de más valor lo antes posible.

Los requerimientos en el tope de la lista no solo serán priorizados, sino que también serán trabajados con el mayor nivel de detalle.

A medida que se avanza en los sprints, los requerimientos pueden ser descartados o agregados en la Product Backlog. Otros pueden subir o bajar en la pila por cambios en su prioridad. Estos cambios son realizados por el PO, que es el único responsable de gestionar la lista.

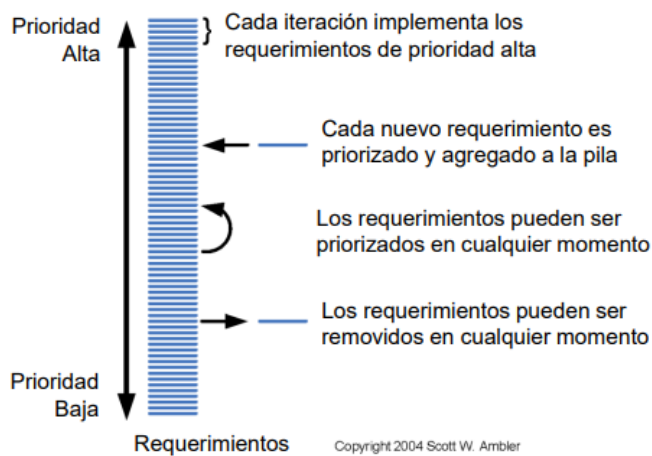
En ágil el cliente es el encargado de la priorización -> se construye el producto deseado.

En el enfoque ágil, a diferencia del tradicional, no se utiliza la ERS para persistir los requerimientos a lo largo del ciclo de vida del producto. En cambio, se utiliza el Product Backlog. La ERS contiene requerimientos ya implementados, en cambio, en el Product Backlog se eliminan los requerimientos ya implementados ya que estos persisten en el producto funcionando.

Los ágilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas

Gestión Ágil de Requerimientos de Software

Los requisitos cambiantes son una ventaja competitiva
si puede actuar sobre ellos



Requerimientos Just in Time

El producto no va a estar especificado 100% desde el principio, **vamos a ir encontrando requerimientos y describiéndolos conforme haga falta.**

Este **concepto evita desperdicios que vendrían de asumir que se conoce todo el producto desde un principio**, lo que incorporaría funcionalidades que el tiempo demostraría como inútiles.

Lo que nosotros tenemos que entregarle al cliente es valor de negocio, no características de software. El software es el medio por el cuál nosotros le entregamos valor de negocio.

METODOLOGIA TRADICIONAL vs METODOLOGÍA ÁGIL

	TRADICIONAL	ÁGIL
Prioridad	Cumplir el plan	Entregar Valor
Enfoque	Ejecución ¿Cómo?	Estrategia (¿Por qué? ¿Para qué?)
Definición	Detallados y cerrados. Descubrimientos al inicio	Esbozados y evolutivos. Descubrimiento progresivo
Participación	Sponsors, stakeholder de mayor poder e interés.	Colaborativo con stakeholders de mayor interés (clientes, usuarios finales)
Equipo	Analista de negocios, Project Manager y Áreas de Proceso.	Equipo multidisciplinario.
Herramientas	Entrevistas, observación y formularios	Principalmente prototipado. Técnicas de facilitación para descubrir.
Documentación	Alto nivel de detalle. Matriz de Rastreabilidad para los requerimientos.	Historias de Usuario. Mapeo de historias (Story Mapping)
Productos	Definidos en alcance	Identificados progresivamente
Procesos	Estables, adversos al cambio.	Incertidumbre, abierto al cambio

La diferencia más marcada entre la gestión tradicional y la gestión Ágil se encuentra en la triple restricción

Sobre el valor: Valor es la obtención de beneficio, ya sea tangible o intangible. El valor lo asociamos a la utilidad, beneficio o satisfacción de los usuarios finales.

TIPOS DE REQUERIMIENTOS ORDENADOS POR PRIORIDAD

- **DE NEGOCIO:** Son los de más alto nivel, nos referimos a cuales son los requerimientos en términos de la visión del negocio
- **DE USUARIO:** Tienen que ver concretamente con los requerimientos de usuario final
- **FUNCIONALES:** Podríamos relacionarlo 1 a 1 con los casos de uso en la gestión tradicional
- **NO FUNCIONALES:** Suelen ser los más ocultos, que el cliente no tiene en claro y debo especificarlos. Un requerimiento no funcional puede hacer que el software que estoy desarrollando no sirva (IMPORTANCIA FUNDAMENTAL)
- **DE IMPLEMENTACIÓN:** Se suele decir que están dentro de los no funcionales.

La idea de la clasificación: entender las necesidades del usuario y las restricciones del contexto en el que se utiliza el software.

Al enfocarse en los requerimientos del dominio del problema y diferenciarlos de los requerimientos específicos del software (del dominio de la solución), los equipos de desarrollo pueden tener una visión más clara de lo que el software debe hacer y enfocar su esfuerzo en las características esenciales para cumplir con los objetivos del proyecto.

En otras palabras, la gestión ágil de requerimientos implica trabajar junto a técnicos y no técnicos entendiendo las necesidades y el negocio para construir un software que de valor. Se descubre como satisfacer las necesidades de los usuarios en compañía de estos.



Sobre la captura de los requerimientos:

Fuente de requerimientos: para el agilismo, la mejor forma de extraer los requerimientos es conversando cara a cara con el cliente. **Se descarta la ERS de las metodologías tradicionales a favor de la disponibilidad para resolver dudas.** Que se descarte la ERS **no implica que no se documenten ni detallen los requerimientos**, sino que se prioriza la correcta obtención de los mismos por sobre la generación de una documentación que no refleje la dinámica de la extracción de los requerimientos en las metodologías ágiles.

Cambia el momento de la captura de los requerimientos:

- **En el enfoque tradicional**, los requerimientos se definen **al principio del proyecto**.
- **En el enfoque ágil**, **los requerimientos son relevados continuamente durante todo el proyecto**. Se inicia la construcción del software sin que los requerimientos no están completamente definidos.

Principios Ágiles (de los 12 principios) relacionados a los Requerimientos Ágiles

- 1) Prioridad: satisfacer al cliente desde lanzamientos tempranos.
 - a. 2 semanas a un mes como duración del reléase (1)
- 2) Recibir cambios de requerimientos simple, incluso en etapas finales (2)
- 3) Técnicos y no técnicos trabajando en conjunto en todo el proyecto (4)
- 4) El medio de comunicación por excelencia es cara a cara (6)
- 5) Los mejores diseños surgen de equipos autoorganizados (11)

User Stories – tomado del práctico

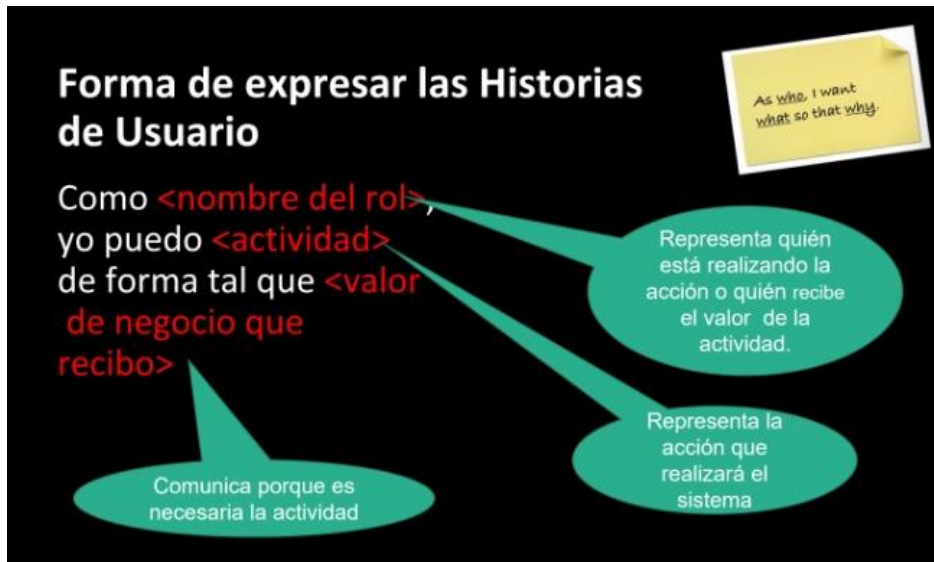
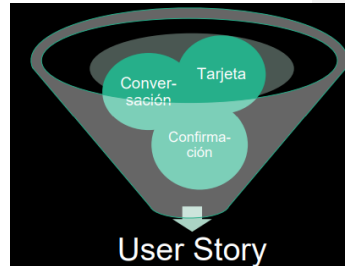
No es una especificación de requerimientos, ya que está a un nivel más alto de abstracción, menos técnico que los reqs.

Requerimientos detallados → casos de uso, no user stories.

Partes de una US:

Las 3 C:

- **Conversación**
 - Las US vienen de la charla.
- **Confirmación**
- Card - tarjeta:
 - **Como rol quiero actividad para valor de negocio**
 - Esta es la famosa frase verbal: una forma de resumir o identificar lo que expresa la user
 - Respecto al rol: no solo poner usuario -> muy general
 - Nunca ¡ nunca pones quiero poder
 - O quiero o puedo



- Nombre
- Actividad
- Valor
- Rol
- Necesidad del rol y el para que
- **Los criterios de aceptación:**
 - Son la forma que tiene el PO de determinar cuando se está cumpliendo, según su criterio, esa US.
 - Los escribe el PO
 - Ayudan a que los PO respondan lo que necesitan para crear US que provean valor (requerimientos funcionales mínimos)
 - Pueden expresarse como RNF.

- Ayudan a que el equipo tenga una visión compartida de la US
- Ayudan a derivar pruebas
- Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US
- Definen los límites para una User Story
- X ejemplo:
 - **Frase verbal:** Como conductor **quiero** buscar un destino a partir de una calle y altura **para** llegar a destino.
 - **Criterios de aceptación:**
 - La altura de la calle debe ser un número natural.
 - La búsqueda no puede demorar más de un minuto.
- Son independientes de la implementación
- Son de un nivel relativamente alto: los escribe el PO.
- Generales, no vamos a nivel implementación. Digo “tabla” en vez de “excel”.
- si son buenas :
 - definen intención > solución
 - independientes de la implementación
 - relativamente de alto nivel: nivel usuario, no nivel sistema
-
- **Las pruebas de aceptación de usuario / pruebas de usuario:**
 - Las escribimos a partir de los criterios de aceptación.
 - Son definiciones de pruebas que va a hacer el usuario para determinar si se cumple o no se cumple con el criterio de aceptación.
 - No son el diseño de una prueba de usuario, sino un punto de partida **para poder diseñar los casos de prueba posteriormente.**
 - Para el ejemplo anterior:
 - Probar buscar un destino en un país y ciudad existente pero en una calle inexistente (falla).
 - Probar buscar un destino en un país y ciudad existentes, de una calle existente y a una altura existente (pasa).
 - Probar buscar un destino en un país existente, en una ciudad inexistente (falla).
 - Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).
 - Defino una prueba para determinar si una funcionalidad funciona o no funciona.
 - Siempre arrancas con probar y nunca pones “el sistema valida tal cosa”... no es técnico ni muy descriptivo

User Stories – Teórico

Son una técnica para capturar requerimientos a un nivel de abstracción muy alto, donde se formula la necesidad como **una descripción corta de la funcionalidad esperada por el cliente.**

Lo de la abstracción: es un requerimiento a nivel de usuario, no al nivel de sistema.

A diferencia de los Casos de Uso, no son especificaciones detalladas de los requerimientos.

Son expresión de intención: necesito esto.

Son descartables, no requieren mantenimiento.

Sirven de entrada para la documentación.

Las US son Multipropósito:

- Son:
 - Necesidades del usuario
 - Descripción del producto
 - Un ítem de planificación
 - Token para una conversación
 - Mecanismo para diferir al conversar

INVEST Model para la calidad de las US

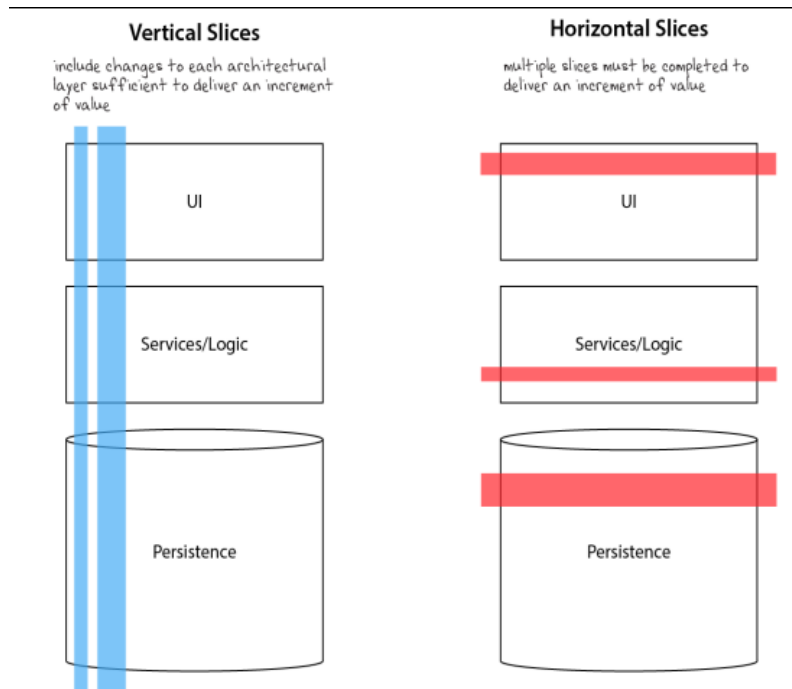
Para verificar que la US esté bien escrita

- **Independiente:** implementables en cualquier orden, no dependen de otra US para empezar.
- **Negociable:** el que, no el como. No dice como implemento sino que hago
- **Valuable** (evaluable): debe tener un valor de negocio / para el cliente
- **Estimable:** debo poder estimar el tamaño de la US puede ingresar a ser desarrollada o no. Si no puedo determinar el peso, **estoy hablando de una Spike**.
- **Small:** deben ser consumidas en una iteración. Sino, no es una US. O se la divide, o replantea o algo.
 - Las iteraciones deben terminar con un valor de negocio incorporado.
- **Testeable:** deben poder correrse las pruebas de usuario. Demostrar que fueron implementadas, es testeable, puedo probarla.

Comentado [BCDL19]: No es un contrato

Porción Vertical.

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de persistencia. Se debe diseñar un pedazo de cada cosa. Abarca todas las capas de la arquitectura.



Product Backlog y US

En el Product Backlog se puede aplicar el mismo trabajo de priorización que se le hace a los requerimientos a las user stories. El Product Owner ubicará en la parte más alta de la pila las US de mayor prioridad. Se podrá cambiar la priorización según lo que indique el PO, así como también agregar o quitar US.

Las 3 C – Partes de una US

Card- Tarjeta

Descripción breve de la Story, planteada con una sintaxis particular:

Como <<nombre del rol>> **yo quiero** <<actividad>> **para** <<valor de negocio>>

- Nombre del rol: quien realiza la acción o recibe el valor de la actividad.
 - Las US ayudan al modelado de roles
- Actividad: acción que realizará el sistema.
- Valor de negocio: el por qué de la actividad. Permite plantear prioridades posteriormente.
- Frase verbal: no corta, forma corta de representar la funcionalidad de la US.

- Criterios de aceptación: criterios sobre la actividad que se necesita implementar. Limitan que debe cubrir la US para que el equipo tenga una mejor visión de lo que representa, facilitando su trabajo.
 - En especial el trabajo de los desarrolladores y testers
 - Aumentan el nivel de detalle.
 - Se escriben con un alto nivel de abstracción: independientes de la implementación.
- Pruebas de aceptación: tareas para probar si una tarea está completa.
 - También definidas a alto nivel.
 - Se plantea si la prueba falla o pasa.

Conversación

Parte más importante de la US, ya que de esta se obtiene la información para definir la US.
La conversación se da entre el equipo de desarrollo y el PO.

Confirmación

Son las condiciones que se deben dar para que una US se de por satisfecha. Conforman a la definición de los criterios y las pruebas de aceptación (que están escritas más arriba en la card porque me confundí xd). El PO decide si las pruebas de aceptación son válidas

Ventajas	Desventajas
● Priorizan la comunicación verbal porque la conversación es lo más importante ● Son comprensibles tanto para el cliente como para el usuario ● Tienen el tamaño ideal para permitir planificar ● Permiten dividir las funcionalidades para el desarrollo iterativo	● No son apropiadas para equipos o proyectos grandes, porque allí se dificulta la comunicación verbal de la que dependen

Pensemos....

¿Qué esta mal en las siguientes historias?

Primordialmente, ninguna cumple con el formato: Como [Rol], quiero [Acción], para [Beneficio].

- El usuario puede ejecutar el sistema en Windows XP y Linux.
 - Describe un RNF. No aporta valor de negocio directo.
- Todos los gráficos se harán con una librería de terceros.
 - Restricción técnica. Supera el nivel de abstracción de usuario de las US.
- El usuario podrá deshacer hasta 50 comandos.
 - Funcionalidad puntual. No dice el para que.
- El software deberá entregarse el 30 de Julio.
 - Restricción de gestión. No revela ningún valor de negocio que se obtenga del comportamiento del software.

- El software deberá estar desarrollado en JAVA
- Restricción técnica también

Y acá que “huele mal”?

- **User Story:**
 - Como usuario, yo necesito almacenar el ID code con 10-14 dígitos, 4 dígitos para el PIN (el cual es opcional) y muchos RFID tags en mi profile, así yo puedo tener muchos log-ins.
- **Criterio de Aceptación**
 - AC1: Manualmente agregar el Id de 10-14 dígitos al profile
 - AC2: Manualmente agregar 4 digit PIN al profile
 - AC3: ver el PIN en pagina de profile
 - AC4: Manualmente agregar 3 RFID tag IDS a mi profile
 - AC5: Ver RFID tags en mi página profile.
 - AC6: SI PIN es requerido, poner una bandera y verificar que el PIN se ha proporcionado

 5 minutos

En la imagen si se cumple el formato de la frase verbal, pero tiene varios errores:

- Define múltiples funcionalidades (id, pin, tags..). No cumple la S de Small.
 - Debe dividirse en US más pequeñas
- Incluye detalles técnicos muy específicos, tanto en los criterios como en la frase verbal.
- Contradicciones en los criterios.

Tamaño de una US:

Es importante determinarlo porque las US deben ser, idealmente, pequeñas (al menos según INVEST).

Hay 3 aspectos claves para determinar el tamaño:

- **Complejidad:** hace referencia a la dificultad inherente de implementar la funcionalidad. Pensamos en los algoritmos, cálculos, campos grandes, validaciones, filtros...
- **Esfuerzo:** es el tiempo en horas de trabajo que requerirá a la implementación.
- **Incertidumbre:**
 - **Técnica:** dudas sobre el dominio de la solución.
 - **De Negocio:** dudas sobre el dominio del negocio.

Story Points

Son unidades de medida que miden el tamaño de un producto.

- No se basan en el tiempo (no usan x ejemplo horas hombre)
- Son relativas → no importa el valor en sí mismo, sino el valor en comparación con otros.
 - Las personas estiman mejor de forma relativa antes que absoluta.

Niveles de Abstracción

- 1) **User Story:** cumple con el criterio de Listo (DoR).
- 2) **Épica:** User Story muy grande. El “muy grande”, el tamaño excesivo, es relativo según si la US entra o en la iteración.

Si la User no entra en la iteración es considerado una Épica y debe dividirse en US más pequeñas.

- 3) **Temas:** son un conjunto de User Stories agrupadas por estar **conceptualmente relacionadas**. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.



Modelado de Roles

Se utilizan las **tarjetas de rol de usuario**. Aquí se anotan las características general del usuario.

Las US relacionan las funcionalidad a quien las van a utilizar.

Técnicas adicionales:

- 1) Personas: análisis exhaustivo de la persona que usará el sistema.
- 2) Personaje extremo: definición muy extrema y general de quien utilizará el software.
- 3) Proxies – Usuario Representantes: es un representante de todos los usuarios.

Criterios de Aceptación

Nota **no obligatoria** que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación.

Es muy útil como parámetro para que los desarrolladores puedan definir si fue cumplimentada o no la funcionalidad -> ósea si el PO la va a aceptar.

Los criterios definen límites (cuando para) para las US, ayudando a que el equipo tenga una visión compartida de la US. Permite también que los desarrolladores y testers creen pruebas.

- Definen una **intención**, **no una solución**

- Son independientes de la implementación. No definen implementación
 - ALTO NIVEL
- Los detalles que se pierden porque son de alto nivel van a
 - Documentación interna de los equipos
 - Pruebas de aceptación automatizadas

Pruebas de aceptación

Son declaraciones de intención de que hay que probar.

En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación.

El PO acepta la US como implementada cuando se cumplen **todas** las pruebas de aceptación.

Definition of Ready - DoR

Medida de la Calidad que determina si la US está en condiciones para entrar a una iteración del desarrollo.

Como mínimo, la US **debe satisfacer el INVEST Model**. El equipo de desarrollo y el PO pueden definir en conjunto otras condiciones para definir si la US está lista para ser implementada en el sprint. La US debería cumplir con **todos** los criterios.

Definition of Done – DoD

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa. Es el momento en que la US cumple con todo lo pactado como para mostrárselo al PO.

En la práctica consta de un **Checklist** de condiciones que deben cumplirse. Puede ocurrir que una organización tenga un estándar de DoD, o que los miembros del equipo deban confeccionar la lista.

Es el DoD lo mismo que pasar los Criterios de Aceptación? NO. Porque los criterios son propios de cada historia, mientras el DoD transversal para todas las US.

Spikes

Son un **tipo especial de US** utilizado para quitar incertidumbre o riesgo a una US, requerimiento o faceta del proyecto

Ejemplos: investigar una tecnología que no conocemos, dudas de cómo plantear alguna interacción del usuario con el sistema, evaluación de librerías, pruebas de concepto...

- **Spike Técnica:** utilizadas para investigar enfoques técnicos en el dominio de la solución.
 - Evaluar performance potencial, tomar decisiones sobre que hacer o comprar, evaluar cierta tecnología
 - cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo
- **Spike Funcionales:** utilizadas cuando hay cierta incertidumbre respecto de como el usuario interactuará con el sistema.
 - Se evalúan mejor con prototipos para obtener retroalimentación.

Sirven para:

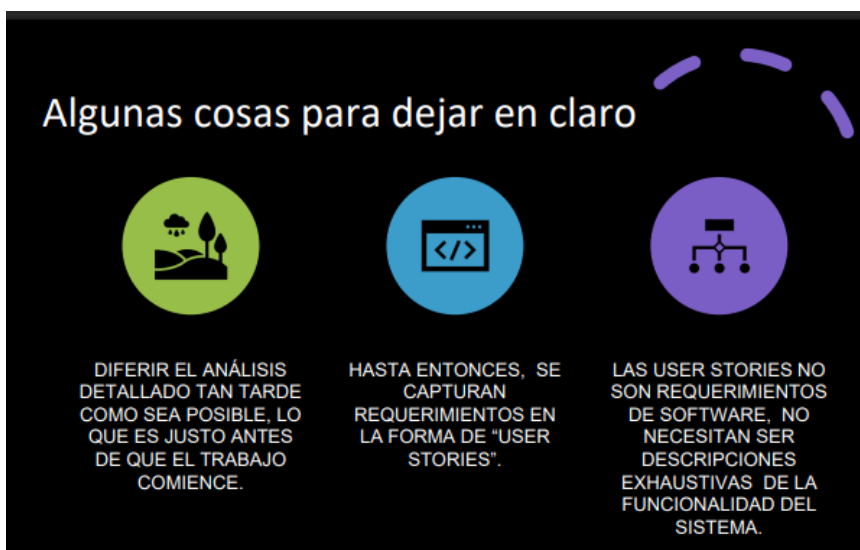
Comentado [BCDL20]: Las US no son requerimientos de software, no necesitan descripciones exhaustivas de la funcionalidad del sistema

- Familiarizar el equipo con una nueva tecnología o dominio desconocido
- Analizar una historia completa antes de fraccionarla
- Ganar confianza frente a riesgos tecnológicos para disminuir la incertidumbre
- Enfrentar riesgos funcionales poco claros

Las Spikes no tienen porque aplicarse siempre. Todas las US tienen algún grado de incertidumbre e implican riesgos, pero no por eso merecen el planteo de una spike. Estas deben surgir ante incógnitas grandes y críticas.

Los spikes deben ser estimables, demostrables y aceptables.

Generalidades sobre las US



Tips:

- La palabra "y" no va en US.
- Usar palabras claras en los criterios de aceptación.
- No olvidar la conversación: invisible pero importante.
- No forzar toda la info en US
- Las US siempre se escriben desde la perspectiva del usuario

Estimación en ambiente ágiles

Estimar es el proceso de obtener una aproximación sobre una medida. Estimar es predecir en un momento de gran incertidumbre.

Una estimación en el contexto del software es una predicción de **cuánto tiempo durará o costará un proyecto.**

Las estimaciones sirven para planificar u obtener respuestas tempranas sobre la factibilidad de lo planificado. Las estimaciones nunca serán perfectas o precisas, por lo que el plan final no tiene porque coincidir con lo estimado. Las estimaciones no son compromisos para el plan.

Son útiles en un principio, porque al iniciar los proyectos predomina la incertidumbre. Mientras más se avanza, más disminuye la incertidumbre. Esto no significa que haya que dejar de estimar, sino que por el contrario, se van haciendo estimaciones cada vez más precisas.

Las Estimaciones son mejores cuando son **relativas**, no absolutas.

Para qué o por qué se estima?

Dos razones principales:

- Predecir completitud
- Administrar riesgos

Orden al Estimar:

(esto ya está en algún otro lado de resumen)

- 1) **El Tamaño:** medida del trabajo a realizar. Lo contamos de alguna forma:
 - a. Contar funcionalidades
 - b. Contar requerimientos
 - c. Contar la cantidad de User Stories
 - i. cuan compleja es, cuanto tamaño conlleva
 - d. Contar la cantidad de Casos de Uso

Nos sirven también datos históricos, por ejemplo, comparar el software a construir con algún software similar que hayamos construido antes.

- 2) **El Esfuerzo:** cantidad de horas lineales que voy a necesitar para construir el software. olo CUENTO, en estimación, cuantas horas lineales de desarrollo voy a necesitar.
 - a. Es frecuente y muy erróneo confundir tamaño con esfuerzo
- 3) **El Calendario:** puedo calendarizar el esfuerzo, planteando una fecha de entrega propuesta.
- 4) **EL Costo/Presupuestos:** lo estimo después del tamaño, fuerzo y tiempo de calendario.

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. **La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo**, de su capacitación, conlleva horas lineales.

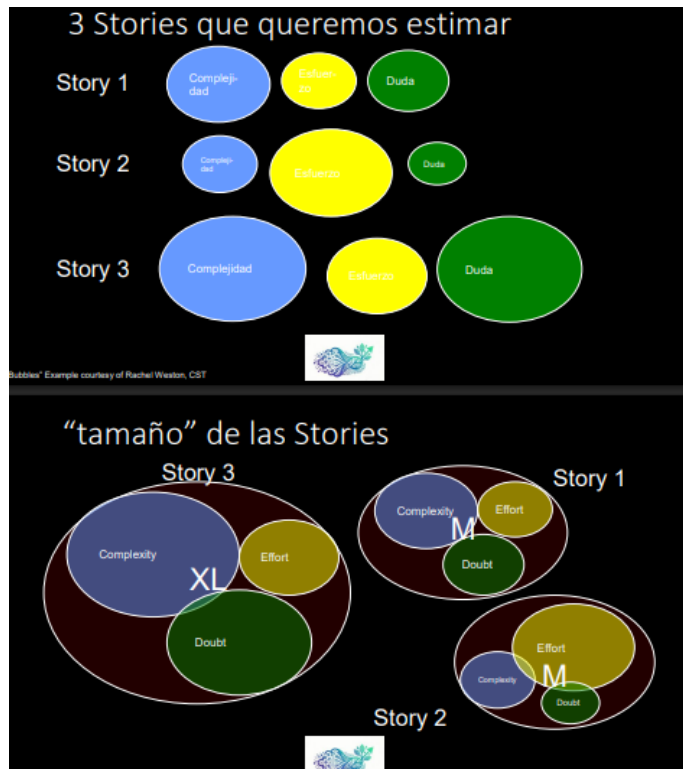
Escalas para definir Tamaño.

- Tamaño por números: 1 a 10
- Talles de remeras: S, M, L, XL, XXL
- Serie 2n: 1, 2, 4, 8, 16, 32, 64, etc.
- Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

(FALTA – MÉTODOS DE ESTIMACIÓN)

Ahora si: Estimación en Ambientes Ágiles

Las User Stories se estiman utilizando una medida **de tamaño** relativo llamada **Story Poinns (SP)**. Estos son una ponderación que se le da a una US **combinando incertidumbre, esfuerzo y complejidad**.



- Complejidad: relacionada a la cantidad de partes y las relaciones entre partes de la US.
- Esfuerzo: son las horas lineales que se requieren para terminar la US.
- Incertidumbre: cuánta información falta para ser más adecuados en la estimación.
 - Incertidumbre Técnica: que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología.
 - Incertidumbre del Negocio: si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.

Se mide con tamaño porque las estimaciones basadas en base al tiempo son más propensas a errores debido a factores externos como las habilidades de las personas, el conocimiento del dominio, la experiencia, etc.

Los SP no son absolutos, sino relativos, por lo que dependerán de cada equipo.

Velocidad

Es una métrica del progreso de un equipo. Se **calcula** como la suma de los Story Points de historias que el equipo completó al 100%.

Recordemos que la US completada es aquella que se ha desarrollado y testeado por completo, superando todos los puntos del checklist DoD y siendo aceptada por el Product Owner.

Como no se suma los SP de las US no terminadas, los SP de las US que después de una iteración quedaron “incompletas” se sumarán repentinamente en la iteración que se termine de completar la

Comentado [BCDL21]: No se estima! Se calcula

US. Entonces, primero percibiremos una velocidad lenta y luego una velocidad rápida, cuando en realidad la velocidad del trabajo fue normal. Entonces, el equipo debería considerar el promedio de las velocidades como información más valiosa que la velocidad de una iteración separada. Así se tendrá un análisis más certero del progreso.

Esta métrica permite retroalimentar y corregir el proceso de estimación, ya que un equipo maduro y con mucho autoconocimiento, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

La velocidad no es una estimación, sino una medida propiamente dicha, obtenida por cálculos.

Poker Estimation – Poker Planning

Método de Estimación (uno de muchos). Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Se pueden usar todas las escalas que vimos antes, pero vamos a usar la de Fibonacci.

- **0:** se desconoce completamente la US, se necesita detallar
- **1:** funcionalidades pequeñas
- **2-3:** funcionalidades pequeñas a medianas
- **5:** funcionalidades medianas
- **8:** funcionalidades grandes, empezamos a plantearnos dividir la US
- **13:** funcionalidad muy grande que no debe ingresar al sprint

Se define una US como **canónica** como patrón de comparación para el resto de las estimaciones. Es conveniente que sea una US simple, de 1, 2 o 3 puntos, y que sea la de menor grado de incertidumbre.

Nota: Hablamos de US pero se pueden estimar también casos de uso, módulos, actividades...

- 1) Defino la lista
- 2) EL equipo acuerda la canónica
- 3) Cada miembro elige un valor para estimar el tamaño de la US
 - a. En comparación con la canónica
 - b. Se tienen en cuenta complejidad, esfuerzo e incertidumbre
- 4) Se hace una puesta en común de los valores estimados
- 5) Se discuten los valores extremos, o sea, la mayor y la menor estimación
- 6) Si existiesen diferencias volvemos al 4
- 7) Si las estimaciones no convergen, entonces se llega a una estimación promedio

Al estimar se debe considerar el DoD → todo lo que implica terminar una US

Ventajas del Poker Planning

- Opiniones desde los distintos puntos de vista de los miembros del equipo multidisciplinario, enriquecedor.
- Los debates generados permiten una mejor comprensión del dominio.
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar a nivel grupal.
- Divertido :)

SCRUM

Scrum es un marco de trabajo (framework) **liviano** que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

A través del rol de Scrum Master, se enfoca un entorno donde:

- 1) El Product Owner ordena el trabajo de un problema complejo en el Product Backlog
- 2) Durante el sprint, el equipo de Scrum convierte una selección del trabajo en un incremento de valor
- 3) El equipo de Scrum y los stakeholders inspeccionan los resultados y realizan ajustes necesarios para el próximo sprint
- 4) Repetir

El marco permite el empleo de diversos procesos, técnicas y métodos: Scrum no da instrucciones detalladas, sino que guías para las relaciones e interacciones del equipo.

Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo. Scrum se basa en el empirismo, y por ende, en sus 3 pilares (TIA):

- **Transparencia:** El proceso y el trabajo emergentes deben **ser visibles** para **aquellos que realizan el trabajo**, así como para los **recibidores del trabajo**. La transparencia permite la...
- **Inspección:** los artefactos de Scrum y el progreso deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables. La inspección permite la...
- **Adaptación:** si algún aspecto del proceso se desvía de lo aceptable. El proceso aplicado puede y debe ajustarse.

Valores de Scrum: *Compromiso, Enfoque, Apertura, Respeto y Coraje*

Otros valores: la Autogestión del equipo, el profesionalismo y responsabilidad de los miembros, y la diversidad cognitiva y de conocimientos.

- Stakeholders: Interesados como usuarios, clientes, decisores.
- Supporters: líderes y gerentes que apoyan la adopción del Scrum sin ser parte del equipo, eliminando impedimentos organizacionales y fomentando la autogestión de los equipos.

Equipo Scrum

La unidad fundamental de Scrum es un pequeño equipo de personas, un equipo Scrum. El equipo Scrum no tiene sub-equipos o jerarquías.

Los equipos Scrum son **multifuncionales** porque están formados por miembros poseedores de distintas habilidades que aportan valor.

El equipo scrum es pequeño para favorecer la agilidad, pero lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint. La idea sería 10 o menos personas.

Los equipos pequeños favorecen la comunicación, y quizá sería mejor dividir un equipo Scrum grande en varios equipos Scrum cohesionados.

Todo el equipo de Scrum **es responsable de crear un incremento valioso y útil en cada Sprint**.

Comentado [BCDL22]: Liviano pq define una estructura mínima de reglas, roles y eventos. No impone ni proces ni técnicas

Comentado [BCDL23R22]: Deliberadamente incompleto!!

Roles

Desarrolladores / Product Developers

Son las personas del equipo scrum que se comprometen a crear cualquier aspecto de un incremento útil (funcional) en cada sprint.

Son los profesionales del Equipo Scrum encargados de crear un incremento utilizable del producto en cada *Sprint*.

Las habilidades específicas que necesitan los desarrolladores son a menudo amplias y variarán con el dominio del trabajo.

Es un rol que puede incluir agentes automatizados.

- Son quienes gestionan el Sprint Backlog
- Crean el incremento terminado

Product Owner

Es responsable de maximizar el valor del producto resultante del trabajo del equipo de scrum. Es el responsable de gestionar el Product Backlog donde se priorizan los trabajos.

Busca maximizar el valor a largo plazo para asegurar la sostenibilidad, alineación estratégica y retorno de inversión (ROI) del producto.

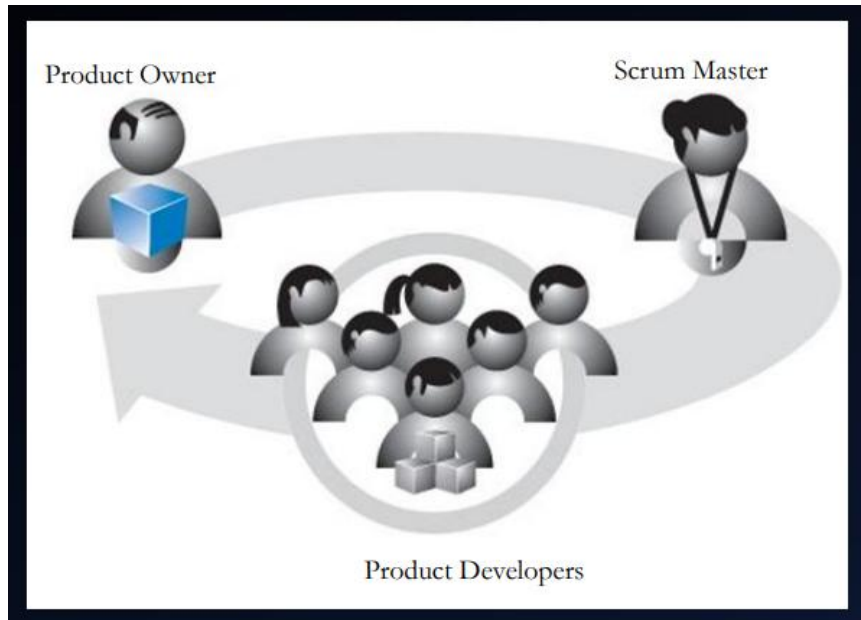
Aseguran que el producto entregue valor al negocio. Proviene del negocio.

Es el representante de los clientes, usuarios e interesados (*stakeholders*) dentro del equipo de desarrollo, responsable de definir la visión del producto y maximizar el valor entregado. Su función principal es gestionar y priorizar el *Product Backlog*, decidiendo qué se construye, en qué orden y con qué criterios de aceptación, actuando como el puente estratégico que traduce las necesidades comerciales y operativas **en requerimientos claros para el equipo técnico**.

Scrum Master

Es responsable de establecer el cumplimiento de Scrum tal como se los define en la guía de Scrum. Ayuda a todos, tanto dentro como fuera del equipo, a entender scrum. Es responsable de la efectividad del equipo.

El **Scrum Master** es el líder que sirve al equipo y el **facilitador** del marco de trabajo Scrum. Su rol principal es asegurar que los valores, principios y prácticas ágiles se comprendan y apliquen correctamente, trabajando activamente para **eliminar impedimentos u obstáculos** que frenen el avance del equipo de desarrollo. A diferencia de un director de proyecto tradicional, no asigna tareas ni ejerce autoridad jerárquica; en cambio, protege al equipo de distracciones externas, fomenta un entorno autogestionado de alto rendimiento y colabora estrechamente con el Product Owner para optimizar la gestión del backlog y la entrega de valor.



Eventos Scrum

- Todos los eventos están diseñados para permitir la transparencia
- Todos los eventos son una oportunidad para inspeccionar, y posteriormente adaptar, artefactos de scrum.

Sprint

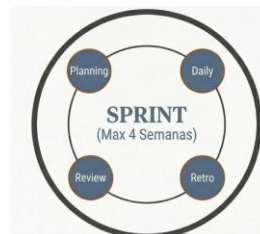
Son **eventos de longitud fija** (recordemos el triángulo invertido) de un mes o menos. Un nuevo sprint comienza inmediatamente después de la conclusión del sprint anterior.

Los Sprints son cortos para evitar la obsolescencia del objetivo y el aumento de la complejidad y los riesgos.

El Sprint es un contenedor para el resto de los eventos (los otros 4).

Durante el sprint:

- No se hacen cambios que pongan en peligro el objetivo del sprint
- Como el tiempo es fijo, el alcance es renegociable



Sprint Backlog (no es un evento)

Define tres aspectos críticos:

- 1) **El por qué: sprint goal.** Se define el objetivo del sprint entre el equipo y los interesados, planteando que podría aumentar el valor del producto.
- 2) **El Qué:** se debate con el PO para seleccionar **que elementos del Product Backlog se incluirán** en el sprint actual. El desafío es cuanto seleccionar.

- 3) **El Cómo:** Para cada elemento de trabajo pendiente de producto (Product Backlog item) seleccionado, los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con la definición de hecho. El **Plan Técnico de Ejecución**

El objetivo de Sprint (**Sprint Goal**), los **elementos de trabajo pendiente** de producto seleccionados para el Sprint, más el **plan para entregarlos** se conocen conjuntamente como el trabajo pendiente de Sprint (**Sprint Backlog**).

Scrum Diario / Daily Scrum

Evento corto (15 minutos max) para **inspeccionar el progreso hacia el Objetivo Sprint** y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado. Está principalmente orientado **para los desarrolladores**. Es una **sincronización táctica**.

Sprint Review

4 horas mensuales como tope.

El propósito de la revisión del Sprint es **inspeccionar el resultado del Sprint** y determinar futuras adaptaciones. Es una sesión **colaborativa**.

El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el Objetivo de Producto. Pero **no es una simple presentación, es una sesión de trabajo**.

El DoD es un criterio que dice cuando una historia está lo suficientemente bien implementada como para entrar al Sprint Review y ser mostrada al PO. Este da la última palabra sobre la aprobación o desaprobación de la US.

Sprint Retrospective

Máximo de 3 horas mensuales.

El propósito de la retrospectiva Sprint **es planificar formas de aumentar la calidad y la eficacia**. El equipo de Scrum inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de Hecho (DoD). Las suposiciones que los desviaron se identifican y se exploran sus orígenes.

El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no fueron) resueltos.

Refinamiento del Product Backlog

I mean, lo dice el nombre

Actividad colaborativa continua entre el Product Owner y los Desarrolladores para desglosar, detallar y estimar los elementos del Product Backlog



Artefactos de Scrum

Representan trabajo o valor. Están diseñados para **maximizar la transparencia** de la información clave.

Cada artefacto está relacionado a un compromiso específico para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso:

- **Product Backlog** → Product Goal
- **Sprint Backlog** → Sprint Goal
- **Incremento** → DoD

Product Backlog:

Pila ordenada de lo que se necesita mejorar o directamente implementar del producto.

Los elementos de trabajo pendiente de producto que puede ser hecho por el equipo de Scrum dentro de un Sprint se consideran listos para su selección en un evento de planificación de Sprint.

El refinamiento de Backlog del producto es el acto de descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños, precisos y transparentes.

Compromiso: Product Goal – Objetivo del Producto.

El objetivo del producto es una meta largo plazo que (Product Goal) **describe un estado futuro del producto que puede servir como objetivo** para el equipo Scrum contra el cual planificar.

“Un producto es un vehículo para entregar valor. Tiene un límite claro, partes interesadas conocidas, usuarios o clientes bien definidos. Un producto podría ser un servicio, un producto físico o algo más abstracto.”

Sprint backlog:

Sprint goal + elementos seleccionados del Product Backlog + plan técnico de ejecución

Porque + que + como

Contiene la lista de elementos del Product Backlog seleccionados para el sprint actual, junto con el plan detallado para completarlos y alcanzar el objetivo del sprint.

Compromiso: Objetivo del Sprint – Sprint Goal

Es el único objetivo del Sprint.

El Objetivo Sprint, a pesar de ser un compromiso de los desarrolladores, también crea coherencia y enfoque, animando al equipo de Scrum a trabajar juntos en lugar de en iniciativas separadas



Comentado [BCDL24]: Poner en algún lado que los proyectos se terminan, mientras los productos son activos vivos

Incremento

Es un paso hacia el Objetivo del Producto. **Cada incremento es una adición a sus sucesores**, y se verifique que funcione junto a estos. Se pueden crear varios incrementos dentro de un Sprint. La suma de los Incrementos se presenta en la Revisión Sprint apoyando así el empirismo. **El trabajo se considera como parte del incremento cuando cumple con todas las condiciones del DoD.**

Compromiso: Definición de Hecho - Definition of Done

La Definición de Hecho es una **descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas** para el producto. En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento.

La **Definition of Done** es una lista de verificación explícita de criterios de calidad, técnicos y operativos que una Historia de Usuario (o incremento) debe cumplir para considerarse completamente **terminada**, ósea, apta para considerarse incremento.

Definition of Output Done vs Definition of Outcome Done

- **DoD Output:** Calidad **técnica**. **Se construyó el producto correctamente?**
 - Criterios como el código limpio, el testeado realizado, la documentación, la integración, se cumplen estándares
- **DoD Outcome:** Valor de **Negocio**. **Se construyó el producto correcto?**
 - Criterios como el impacto en el negocio, el retorno de la inversión, el impacto en el comportamiento del usuario
 - Responsabilidad del PO

El **Output debe cumplirse dentro del Sprint** para considerar que lo trabajado conforma un incremento, pero **el Outcome se mide generalmente después del lanzamiento** e **indica un éxito real**: se está aportando valor con lo trabajado.

Característica	Definition of Output Done	Definition of Outcome Done
Asociado a	El Incremento (Calidad)	El Producto (Valor)
Propósito	Debido proceso y estándares técnicos	Validación de beneficios realizados
Frecuencia	Se cumple en cada ítem del Sprint	Se mide antes, durante y tras el release

ñam ñam ñam pixeles

Matriz de Calibración Estratégica			
	Definition of Ready	Definition of Output Done	Definition of Outcome Done
Estatus en el SGEF	Cualidad emergente (Refinement).	Compromiso formal.	Compromiso formal.
Artefacto Asociado	Product Backlog Item (PBI).	Increment.	Product.
Responsabilidad y Decisión	Scrum Team colaborativo (Devs estiman).	Developers (Accountability colectiva).	Product Owner (Final say).
Métrica Típica	Días / Capacidad de comprensión.	Tests, Seguridad, Estándares técnicos.	Telemetría, Comportamiento de usuario, ROI.

Definition of Ready (DoR)

El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint.

Timebox

El **timebox** (o caja del tiempo) es una técnica de gestión que consiste en **asignar un tiempo máximo fijo a una actividad**, evento o tarea.

Existe para que no se pierda más tiempo del necesario en las distintas tareas. Si notamos que no llegamos al final del Sprint, no se lo extiende, se entrega menos (triángulo invertido). Con el timebox buscamos un ritmo sostenible.

Capacidad del equipo en un Sprint

La Capacidad es una métrica utilizada para **determinar cuánto trabajo puedo comprometer un equipo para un determinado Sprint**.

La capacidad se estima y se mide. Una de las **cosas que se hace en el Sprint Planning** es determinar la capacidad del equipo, y entonces, teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.

Niveles de Planificación

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de reléase
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

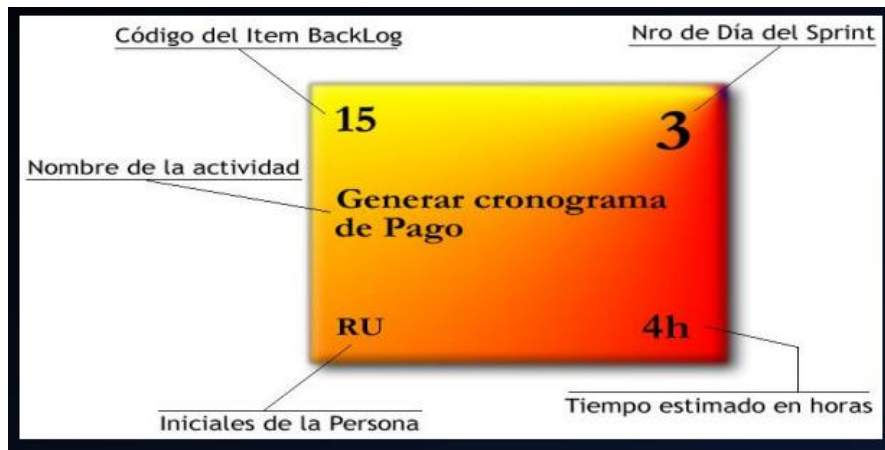


Herramientas de SCRUM

Tarjeta de Tarea

Tarjeta que cuenta con 5 partes:

- 1) Código del Ítem Backlog: identificador.
- 2) Nombre de la actividad: frase verbal.
- 3) Iniciales del responsable de la tarea-
- 4) Número de día del sprint
- 5) Estimación del tiempo para finalizar la tarea



Se puede juntar varias en un **Taskboard**

Taskboard

Taskboard

Story	To Do	In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Code the... DC 4	Test the... SC 6	Code the... D 8
	Code the... 2	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4		Test the... SC 6
As a user, I... 5 points	Code the... 8	Code the... DC 8		Test the... SC 8
	Code the... 4	Code the... 6		Test the... SC 6

Es mejor tener varias en un día que pocas, planteando una buena granularidad. Recordemos que, si son US, es mejor que sean chicas.

Se colocan las tareas a realizar durante el sprint.

Se divide en 5 secciones:

- Story: nombre de la historia.
- To Do: aquí se encuentran las tareas por hacer en el Sprint.
- Work/In Process: tareas que se están realizando
- To Verify: tareas finalizadas pero que se deben verificar.
- Done: tareas terminas → finalizadas y verificadas.

Gráficos del Backlog

Brindan información **sobre el progreso de un Sprint**, un reléase o un producto.

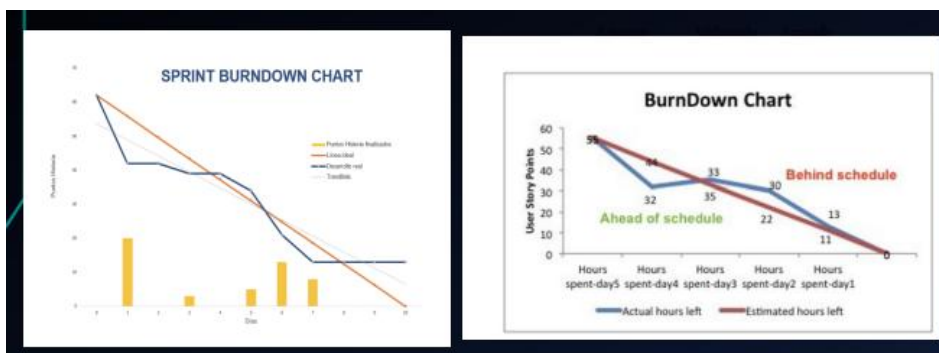
- Backlog de trabajo: cantidad de trabajo que me queda por realizar.
- Tendencia del backlog: compara el trabajo que me queda con la cantidad de tiempo.

Sprint Burn-Down Chart: cuanto queda

Visualiza con una **curva descendente** cuánto trabajo queda para terminar.

- Eje x: tiempo desde que se inició el sprint.
- Eje y: Story Points a realizar en el sprint.

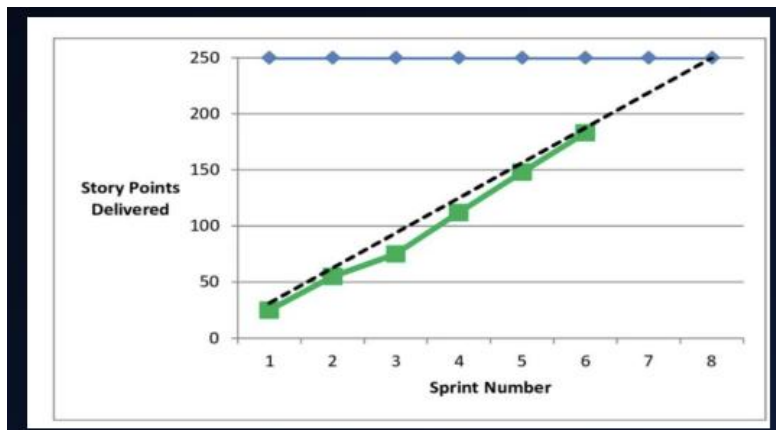
Es descendente porque cada vez se hacen más.



Sprint Burn-Up Chart: cuanto hice

Visualiza con una **curva ascendente** cuánto tiempo se ha completado a lo largo del reléase.

Visualiza los puntos de historia realizados en el sprint.



Combined Burn Chart -> cuanto queda y cuanto se ha realizado

Métricas en ambientes ágiles

Es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (ya sea proceso, producto o proyecto). Son expresadas de tal forma que permiten una medición objetiva de la realidad.

Todas las métricas poseen un costo asociado, debido a que se deben destinar recursos para su planificación.

No entiendo bien que caralho es una métrica pero acá van algunas:

Métricas de Capacidad:

Métrica de Proyecto que mide el compromiso del equipo con un determinado sprint **en horas de trabajo ideales**.

En la PPT las calculan como:

Horas de trabajo disponibles por Día x Días Disponibles de Iteración = Capacidad

WH x DA = Capacidad

RTF – Running Tested Features:

Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (ya sean US, CU, requerimientos) se terminaron y están en ejecución.

El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan: por ejemplo, mide cantidad de US sin considerar sus SP.

Si la curva es constante (llana) o tiene pendiente negativa, entonces indica la existencia de un problema, ya que las características no incrementan o disminuyen en el tiempo

Velocidad

Medida del progreso del equipo. Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.

Se cuentan los story points de las users Stories que están completas, no parcialmente completas .

La Velocidad corrige los errores de estimación.

MVP y sus primos

MVP – Mínimo Producto Viable

Concepto de Lean.

Versión de un nuevo producto que **permite a un equipo recopilar la cantidad máxima** de aprendizaje validado sobre los clientes **con el menor esfuerzo posible**. Está orientada a un subconjunto de clientes potenciales.

Ver lo que la gente realmente hace con el producto es mucho más confiable que preguntarle a la gente qué harían.

La idea del MVP es trabajar con lo mínimo del producto que necesito obtener. Lo mínimo que debería tener el producto para que los (posibles) usuarios del mismo lo puedan evaluar y me puedan decir si el producto les atrae o no.

La versión más básica posible del producto, que cuente con las características esenciales para resolver un problema y satisfacer a los clientes.

La idea sería seleccionar las funcionalidades esenciales del producto, sin implementar funcionalidades extras que no necesiten las esenciales. Por ejemplo para la app de venta de ropa: vender ropa, buscar producto, cargar prenda son US esenciales; pero escribir reseña, pedir devolución, generar QR para compartir item no lo son realmente.

El objetivo del MVP es comprobar que la hipótesis del producto a construir funciona o si tengo que hacer cambios. La idea principal no es vender nuestro MVP, sino validar la hipótesis mediante el feedback.

Puede suceder, como bien dijimos, que así como está definida la hipótesis sirva (todos los clientes/usuarios que vean ese MVP se sientan a gusto y expresen que lo necesitan) o puede ocurrir que cada potencial usuario de ese MVP cuando lo vea pida funcionalidades diferentes.

Si con cada cliente potencial o con cada usuario que prueba la MVP recibo un feedback distinto, quiere decir que no está tan claro el mercado en el cual este producto va a funcionar.

Otra alternativa que podría suceder es que, en algún punto, la hipótesis planteada tenga algunas variaciones. Y en este contexto, la hipótesis puede moverse o modificarse según la exigencia del mercado. Mi MVP se va moviendo hacia el foco en el cual los clientes se van a interesar. Si bien el foco elegido en la hipótesis no es erróneo, algunas características del mismo no están tan bien, por lo tanto, las desviamos.

Si el MVP está muy alejado de lo deseado, se crea un segundo MVP, y así en un circuito de retroalimentación hasta encontrar el producto que se debe construir.

La idea del MVP sería observar el comportamiento real de los clientes con el producto.

A pesar de ser mínimo, el MVP debe tener el valor suficiente para que las personas estén dispuestas a usarlo.

Tipos de Hipotesis que se pueden validar:

- **Hipótesis de Valor:** prueba si el producto realmente está entregando valor a los clientes.
- **Hipótesis de Crecimiento:** prueba como nuevos clientes descubrirán el producto.

MMF – Mínima Característica/Feature Comercializable

Se enfoca en definir los elementos mínimos que deben estar presentes en un producto o servicio para que este sea comercializable.

Ya no estoy parado en una hipótesis para validar si el producto tiene mercado o no, sino que estoy definiendo cual es la pieza mínima que voy a tener que construir y comercializar para que el producto sea rentable y funcione como una beta.

También es importante construirlo en cuestiones mínimas para sacarlo rápidamente a la venta, ya que cuanto más tiempo pase más riesgo corro de que mi idea sea implementada antes -> productos novedosos para meter en el mercado.

El objetivo es asegurarnos de que este primer MMF sirve o no. Con la diferencia de que ahora hablamos de una Feature, algo mucho más chiquito que el MVP. **Es lo mínimo que puedo sacar a la venta.**

Es parte de un MMR o MMP.

MMP – Minium Marketeable Product – Lanzamiento Comercial Mínimo

Primer reléase de un MMR, dirigido a primeros usuarios. Se hace foco en las características clave que satisfarán a este grupo clave-

MMR – Minimum Marketable Release

Nuevo reléase/incremento de un producto que agrega el conjunto de características más pequeño posible. Es el incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.

MMP = MMR1.

MVF – Minimal Viable Feature

Es una versión mini del MVP. Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica. El MVF proporciona un valor claro a los usuarios.

Se trata de la característica mínima que se puede construir e implementar rápidamente, utilizando recursos mínimos, para probar la utilidad de la misma.

Errores comunes

- Confundir un MVP (que se enfoca en el aprendizaje y en validar la hipótesis) con MMF o MMP que ambos se enfocan en tener un valor de retorno, en ganar.
- Enfatizar en el aspecto mínimo perdiendo el punto de lo viable.
- Entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto no está bien.

Gestión de Configuración de Software

<<<SCM>>>

El software no es solo un producto aislado: debemos considerar todo el entorno, las herramientas, procesos, etc. Pero sabemos que el software siempre cambia, tanto durante su desarrollo como durante su uso.

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para **no perder la trazabilidad de los cambios que se incorporan a cada versión**.

La Evolución del software, ya sea por cambios en el negocio y surgimiento de nuevos requerimientos, por cambios asociados a los productos, por cambios en el presupuesto, por defectos

a corregir o por oportunidades de mejora, **siempre debe hacerse intentando mantener la integridad del producto de software**. Para eso está SCM.

Resumen: Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para **no perder la trazabilidad** de los cambios que se incorporan a cada versión.

Problema: el software es muy fácil de modificar, demasiado. Con un solo click se pueden eliminar meses trabajo. Al **asegurar la integridad del producto**, evitamos perder el control ante los cambios.

Gestión de Configuración:

Disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, que **surge para enfrentar la crisis de problemas de software**, aplicando dirección y monitoreo a:

- Identificar características técnicas y funcionales de ítems de configuración
- Documentar características técnicas y funcionales de ítems de configuración
- Controlar los cambios de esas características identificadas y documentadas
- Registrar y reportar estos cambios
- Verificar correspondencia con los requerimientos (trazabilidad)

Su propósito es resolver problemas de diferente índole, a través del establecimiento y el mantenimiento de **la integridad del producto de software a lo largo de todo su ciclo de vida**.

Problemas:

- Pérdida de componentes
- Pérdida de cambios y versiones enteras
- Regresión de fallas
- Mantenimiento duplicado
- Superposición de cambios
- Cambios no validados

La idea es identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

La integridad nos ayuda a garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.

La integridad del producto de software se mantiene cuando:

- **Satisface las necesidades de usuario**: hace lo que el usuario espera que haga.
- **Permite la rastreabilidad** durante todo su ciclo de vida: existen conexiones entre los ítems de configuración que permiten analizar donde impactarán los cambios en un ítem.
- **Satisface los criterios de performance** y los RNF
- **Cumple con expectativas de costo**: desarrollo redituable.

Ítem de Configuración

Pieza que **forma parte del producto y puede almacenarse en un repositorio**. Los ítems pueden sufrir cambios, por lo que es importante conocer su estado y evolución a lo largo del ciclo de vida.

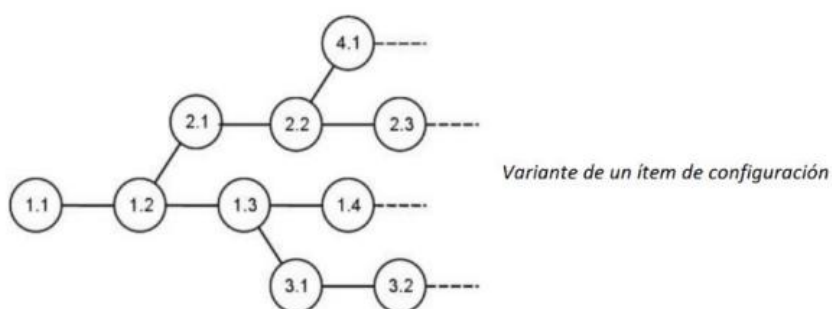
Versión

Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Las versiones se identifican unívocamente.

La gestión de configuración permite a un usuario especificar configuraciones alternativas del sistema de software mediante la selección de las versiones adecuadas. Las versiones, al ser un punto particular en el tiempo de ese ítem de configuración, son un estado.

Control de Versión

Hace referencia a la evolución de un único ítem de configuración o de cada ítem de configuración por separado



Configuración

Conjunto de todos los ítems de configuración con su versión específica. Es la sumatoria de todos los ítems de configuración **en un momento dado.**

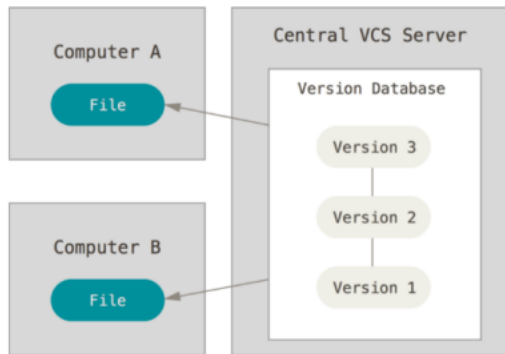
Repositorio

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones.

Los repositorios también se utilizan para hacer evaluación de los cambios propuestos. Tienen una estructura para mantener el orden y la integración.

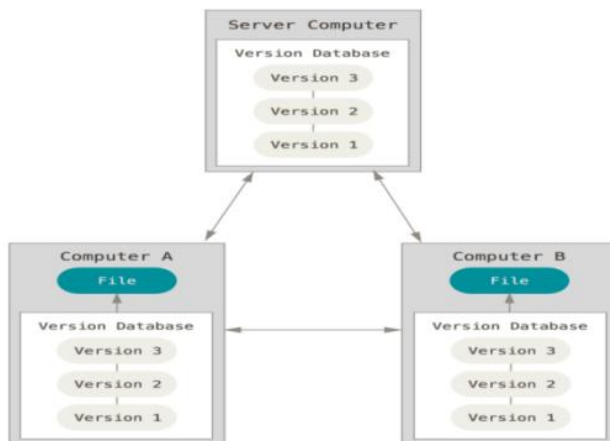
1) Repositorios Centralizados:

- a. Un servidor contiene todos los archivos con sus versiones
- b. Más control, pero se cae el servidor y todo cae con él



2) Repositorios Descentralizados:

- Cada cliente tiene una copia igual del repositorio completo.
- Si falla el servidor, tengo en otro dispositivos una suerte de backup.
- Desventaja: es que es más complicado llevar un control sobre el mismo.



VSC - Version Control System: herramienta de software que registra y gestiona los cambios realizados en el código fuente o en archivos de un proyecto a lo largo del tiempo.

Rama – Branch

Conjunto de ítems de configuración (con sus correspondientes versiones) que permiten bifurcar el desarrollo de un software, por ejemplo para:

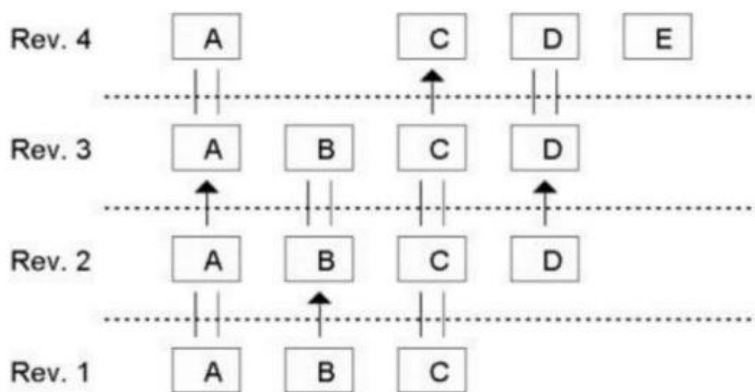
- Experimentación
- Resolución de errores en el desarrollo
- Desarrollar un mismo software para distintas plataformas

Integración de Ramas - Merge

Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas.

Una buena práctica es mantener la rama principal como la versión estable, y una vez seguro de los cambios, fusionar los cambios hacia ella.

Líneas Base



Es un conjunto de ítems de configuración que han sido contruidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos.

Una línea base es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.

Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.

Para cambiar o aprobar una línea base existen protocolos formales planteados por el **Comité de Cambios**, que permite definir el procedimiento a seguir para manejar peticiones de cambios.

La Línea Base sirve:

- como **punto de referencia**
- sirven para hacer rollback, revirtiendo cambios recientes en un sistema hasta un punto aprobado y estable.

Tipos de línea base

- 1) **Operacionales:** contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad bien definido.
- 2) **De especificación o documentación:** son las primeras líneas base, ya que no cuentan código. Podría contener el documento de especificación de requerimiento. Son de requerimientos, diseño.

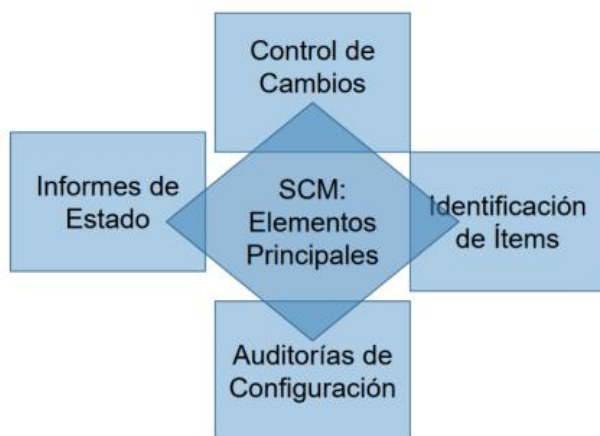
Ejemplo



Planificación de la SCM:

Este plan se debe confeccionar al inicio de un proyecto, y existen diferentes estándares donde se expresa cómo planifica. Se plantea:

- Reglas de nombrado de los ítems de configuración
- Herramientas para utilizar en SCM
- Roles e integrantes del Comité de Control de Cambios
- Procedimientos formales de cambios
- Procesos de auditoría
- Estructura del Repositorio
- SCM para software externo
- Tipos de documentos admitidos
- Registros que deben mantenerse



Actividades relacionadas al SCM

Conjunto de tareas que componen el Plan de Gestión de Configuración

La metodología ágil aplica todo SCM menos las auditorías.

Identificación de Items de Configuración

Consiste en darle a cada ítem una identificación única mediante reglas de nombrado y versionado organizando:

- la estructura del repositorio
- la ubicación de cada ítem en él
- los componentes de la línea base

Establecer los ítems que se deben mantener crea un camino entre las distintas etapas del desarrollo.

Clasificación de los ítems:

- Ítems del producto: elementos que duran toda la vida útil del software
- Ítems del proyecto: elementos que duran lo que dura el proyecto.
- Ítems de Iteración: elementos que duran solo un sprint/ciclo.

Luego de identificar los ítems, **se debe armar la estructura del repositorio** con nombres representativos. Luego se vinculan los ítems de configuración con el lugar físico del repositorio donde se van a almacenar.

Control de Cambios

Para definir una línea base como actualización, y por ende, reemplazo de la línea base anterior, se tiene que pasar un proceso formal de **control de cambios**.

Este proceso es llevado a cabo por el **Comité de Control de Cambios**, el cual se reúne para autorizar la creación y cambios de la línea base. El líder del proyecto, el analista, el arquitecto e incluso el cliente pueden constituirlo: cualquier interesado en conocer y evaluar el cambio

Etapas:

- 1) Se recibe una propuesta de cambio sobre la línea base.
 - a. No sobre un ítem puntual, sobre la línea entera
- 2) El comité realiza un análisis del impacto del cambio para evaluar el esfuerzo técnico y el impacto global sobre el producto.
- 3) Si se autoriza la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones y los criterios para revisar y auditar.
- 4) Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente.
- 5) Se notifican los cambios a los interesados

Auditorías de Configuración

Controles independientes y objetivos realizados durante el desarrollo por un **auditor externo al equipo**. Este auditor analiza las líneas base para ver el estado del software y si se cumple con las reglas de SCM.

Revisan el cumplimiento del Plan de Gestión de Configuración y complementan la revisión técnica, ayudando a la calidad e integridad.

Trazabilidad / Rastreabilidad de los requerimientos

Establecer una correspondencia de relación entre ítems de configuración de diferentes niveles de abstracción en el proceso de desarrollo de software.

Una traza puede ser constituida por:

- el requerimiento
- sus modelos de análisis y de diseño
- la implementación en código
- los casos de prueba para validar y verificar el requerimiento

La trazabilidad permitirá analizar como una solicitud de cambio en un requerimiento podría impactar al sistema.

Los modelos de calidad exigen que la trazabilidad requerimiento/código sea bidireccional.

Procesos a los que sirve:

- 1) **Validación:** se encarga de asegurar que cada ítem de configuración **resuelva el problema apropiado**, es decir, **lo que el cliente necesita**.
- 2) **Verificación:** implica asegurar que **el producto cumple con los objetivos definidos en la documentación de líneas base**. **Las funcionalidades son llevadas a cabo con éxito, superando los tests**.

Tipos de Auditorías

- 1) **Auditoría Física** o de contingencia: se realiza primero. **Se validan los ítems de configuración** de la línea base contra el plan de gestión de configuración, **asegurando que la documentación que se entregará es consistente con el código desarrollado**. Si esto no pasa, la auditoría funcional no se realiza
 - a. **Verificación**
- 2) **Auditoría funcional de configuración:** **audita la trazabilidad de requerimientos**, controlando **que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos**. El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS.
 - a. **Validación**

Comentado [BCDL25]: Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados. • Lo que se audita es una línea base

Por qué no se audita en ambientes ágiles

La gestión de configuración en ambientes ágiles **se trabaja con integración continua** (CI - Continuous Integration), un servidor donde se testea automáticamente el código y luego se integra en producción continuamente. Un repositorio, un esquema de nombrado son cosas que sí se utiliza en ágiles, mientras que la auditoría y el comité de control de cambios durante el sprint, pueden obviarse.

Informe de Estado

Provee un mecanismo para mantener un registro de como evoluciona el sistema y dónde está ubicado el software en comparación con lo publicado en la línea base.

Permite mantener a todo el equipo informado sobre la última línea base para que se trabaje sobre la última versión en lugar de una desactualizada.

Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Como son muchos datos, la automatización es muy útil.

El objetivo principal es comunicacional: asegurarse que la información de los cambios llegue a todos los involucrados.

Permite responde a preguntas como:

- ¿Cuál es el estado del ítem?
- ¿La propuesta de cambio fue aceptada o rechazada por el comité?
- ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
- ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

EXTRAS

Preguntas Interesantes:

¿Si elegimos un ciclo de vida iterativo e incremental, cuál sería la diferencia entre plantear una iteración en un proceso empírico y en un proceso definido. Justifique cada respuesta?

La diferencia principal radica en **cómo se gestiona la incertidumbre y cómo se planifica el trabajo de cada iteración**: mientras que en un **proceso definido** **cada iteración se ejecuta siguiendo un plan predictivo y predeterminado**, en un **proceso empírico** **cada iteración se basa en la inspección del resultado real anterior** para adaptarse y decidir el siguiente paso.

Dificultades Accidentales

Son **las complicaciones secundarias vinculadas a cómo se construye el software**, las cuales fueron solucionadas con el tiempo. Entre ellas están los lenguajes de programación de alto nivel, que independizan la programación del hardware; el tiempo compartido, que reduce los tiempos de respuesta del sistema para no desviar el foco de trabajo; y los entornos de programación unificados, que integran librerías, marcos de trabajo y formatos en un solo lugar.

Dificultades Esenciales

Son los **desafíos propios e inevitables de idear software** (conceptos abstractos, estructuras de datos y algoritmos). Comprenden cuatro propiedades fundamentales:

- **Complejidad**: Es **inherente al software** debido a su abstracción. **A medida que el sistema crece, su complejidad aumenta de forma exponencial** y no lineal, lo que genera fallas en el producto, retrasos de fechas, sobre costos y problemas de comunicación en el equipo.
 - **Conformidad**: El software no responde a leyes físicas naturales, sino que **debe adaptarse obligatoriamente a sistemas preexistentes creados por humanos**, cuyas reglas suelen ser desordenadas, caprichosas y carentes de una lógica perfecta.
 - **Mutabilidad**: A diferencia de los objetos físicos que suelen desecharse para fabricar uno nuevo, **el software está sujeto a constantes cambios y modificaciones a lo largo del tiempo**.
- Invisibilidad: Al carecer de presencia física tangible, el software no se puede representar de forma geométrica, por lo que se recurre a grafos para modelar sus estructuras y relaciones.
- **Invisibilidad**: Al **carecer de presencia física tangible**, el software no se puede representar de forma geométrica, por lo que se recurre a grafos para modelar sus estructuras y relaciones.